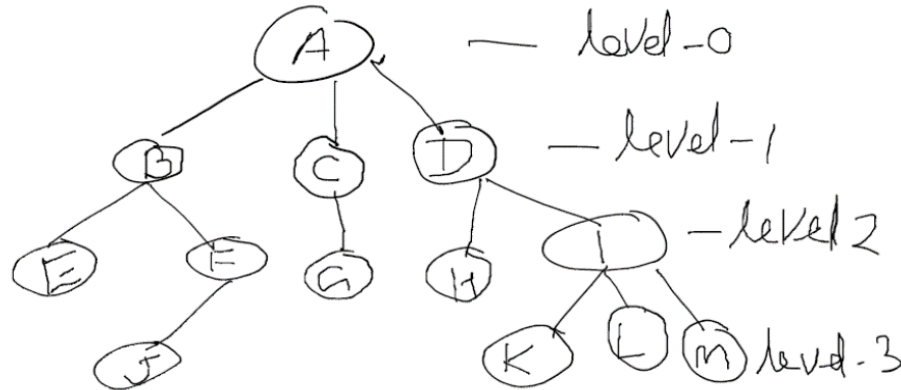


UNIT – 4

TREES

Tree: a tree is a non linear data structure In which items are arranged in a sorted sequence. It is used to represent hierarchical relationship existing amongst several data items. It has a special data item called the root of the tree and its remaining data items are partitioned into number of mutually exclusive (disjoint) subsets, each of which is itself a tree, and they are called sub trees.



Tree terminology: There are number of item associated with the trees:

Root: It is the first node in the hierarchical arrangement of data items.

Node each data item in a tree is called a node.

Degree of a node Degree of a node is the total number of children of that node. For example in the above Tree, the degree of node a is 3 and the degree of node B is 2.

Degree of a tree is the highest degree of a node among all the nodes in the tree..

Terminal nodes, a node with a degree zero is called a terminal node or a leaf node.

Non terminal nodes Any node, except the root node, whose degree is not zero, is called non terminal node

Siblings The children nodes of a given parent node are called siblings.

Level The entire tree structure is labelled in such a way that the root node is always at level zero, then its immediate children are at level 1 and their immediate children are at level 2 and so on up to the terminal nodes. If a node is at level N then its children will be at level N + 1

Path It is a sequence of consecutive edges from the source node to the destination node In the above tree, the path between A and K is given by the node pairs (A, D), (D, I) and (I, K).

Depth It is the maximum level of any node in a given tree . In a tree data structure, the total number of edges from root node to a particular node is called as DEPTH of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be Depth of the tree.

In simple words, the highest depth of any leaf node in a tree is said to be depth of that tree.

Forest It is a set of disjoint trees in a given tree. If you remove its root node, then it becomes a forest in the above tree, There is forest with three trees.

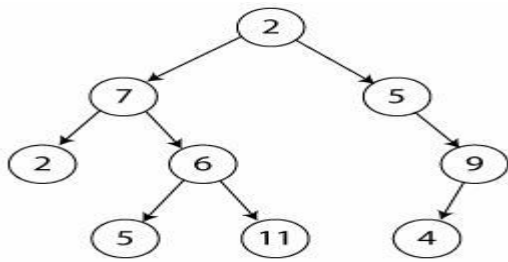
Edge it is a connecting line of two notes.

Ancestor and Descendent If there is a path from node A to node B, then A is called an ancestor of B and B is called a descendent of A.

Height: height of a **tree** is equal to the number of nodes on the longest path from the root to a leaf.

Binary Trees

A binary tree is a finite set of data items, it is a non linear data structure. In a normal tree, every node can have any number of children. Binary tree is a special type of tree data structure in which every node can have a maximum of 2 children. That means, there may be a zero degree node or a one degree node and two degree node. Tree is either empty or consist of a single item called the root and two disjoint binary trees called the left subtree and right subtree.



Binary Trees

Properties of the binary tree

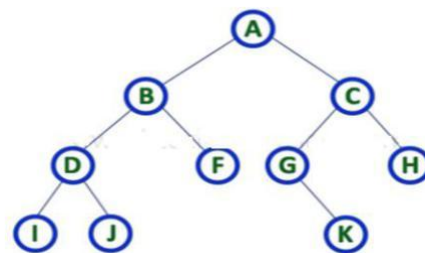
Properties of a Binary tree

- The maximum number of nodes at any level 'L' in a binary tree is 2^L
- The minimum number of nodes in a binary tree of height H is $H + 1$
- The maximum number of nodes in a binary tree of height H is $2^{H+1} - 1$

Representation of a binary tree

A binary tree data structure is represented using two methods. Those methods are as follows...

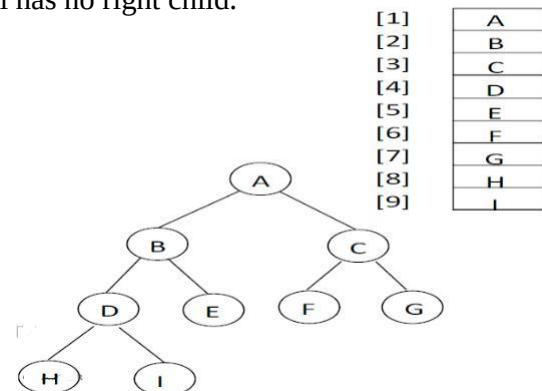
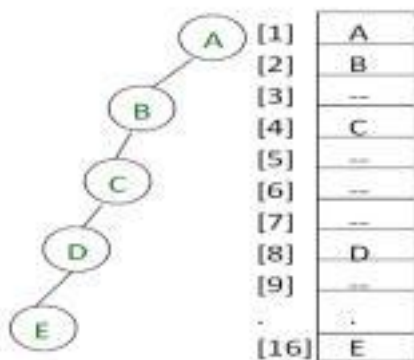
1. Array Representation
 2. Linked List Representation
- o Consider the following binary tree.

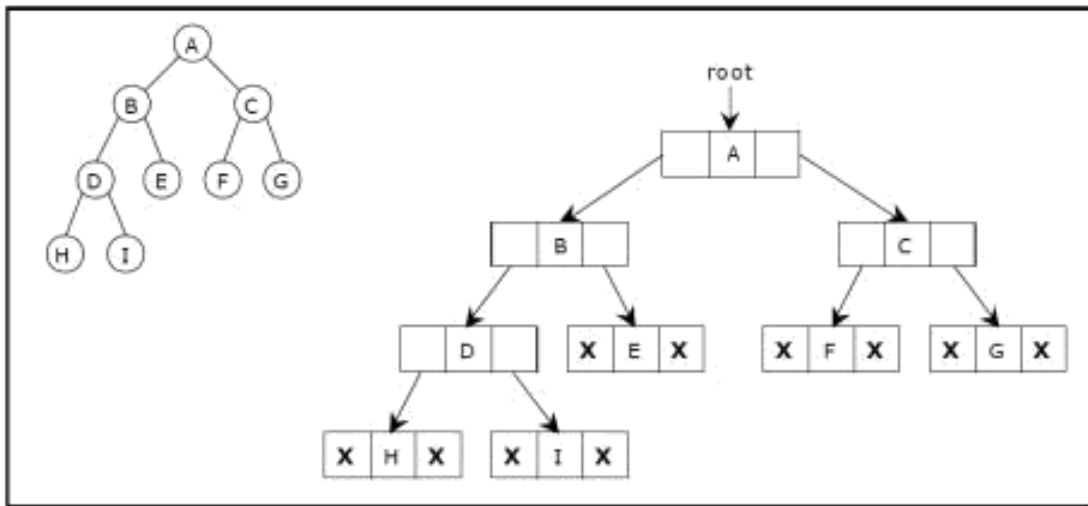


1) Array Representation:

In array representation of binary tree, we use a one dimensional array (1-D Array) to represent a binary tree. To represent a binary tree of depth 'n' using array representation, we need one dimensional array with a maximum size of array. A complete binary tree with n nodes (depth = $\log n + 1$) is represented sequentially, then for any node with index i, $1 \leq i \leq n$, we have:

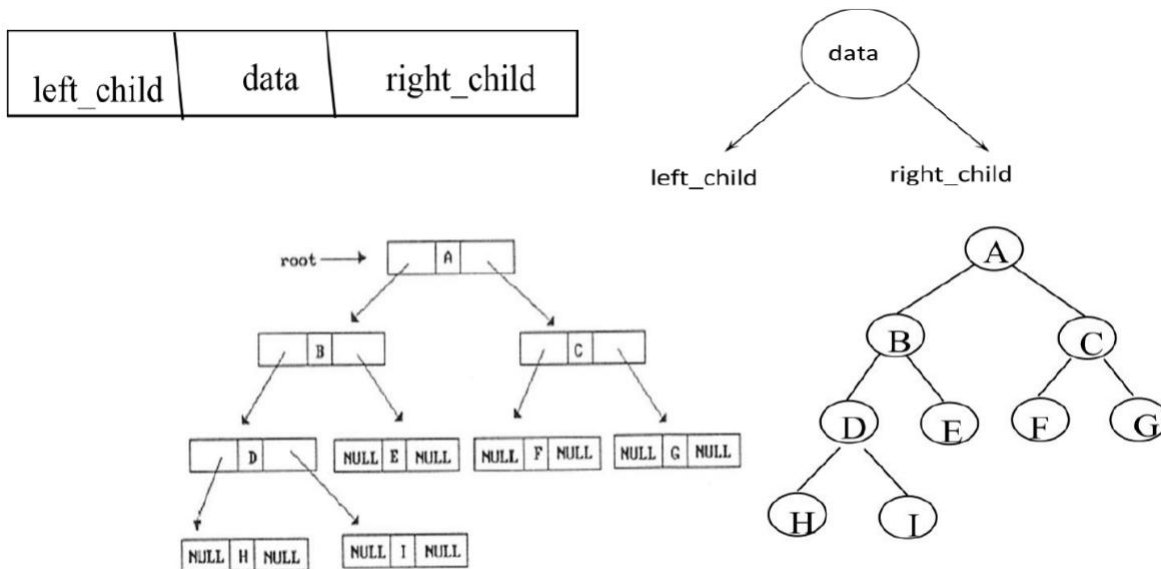
- a) parent(i) is at $i/2$ if $i \neq 1$. If $i=1$, i is at the root and has no parent.
- b) left_child(i) is at $2i$ if $2i \leq n$. If $2i > n$, then i has no left child.
- c) right_child(i) is at $2i+1$. If $2i+1 \leq n$. If $2i+1 > n$, then i has no right child.



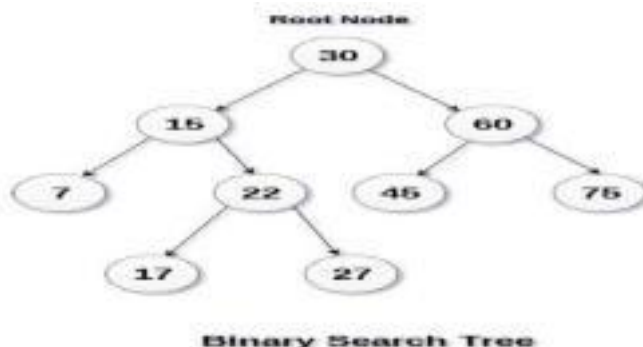


Disadvantages:(1) waste of space (2) insertion/deletion problem

2. Linked Representation We use linked list to represent a binary tree. In a linked list, every node consists of three fields. First field, for storing left child address, second for storing actual data and third for storing right child address. In this linked list representation, a node has the following structure...

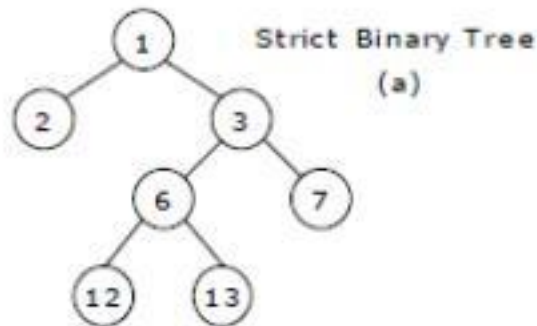


Binary Search tree: Binary search tree is a non-linear data structure in which one node is connected to n number of nodes. It is a node-based data structure. A node can be represented in a binary search tree with three fields, i.e., data part, left-child, and right-child. A node can be connected to the utmost two child nodes in a binary search tree, so the node contains two pointers (left child and right child pointer). Every node in the left subtree must contain a value less than the value of the root node, and the value of each node in the right subtree must be bigger than the value of the root node.



Strictly Binary tree:

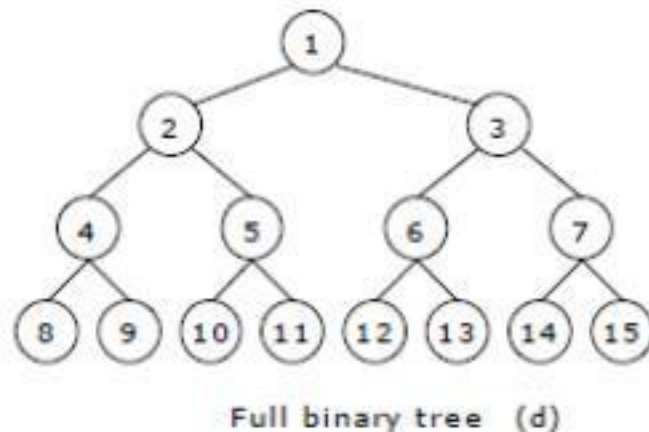
- If every non-leaf node in a binary tree has nonempty left and right subtrees, the tree is termed a strictly binary tree. A strictly binary tree with n leaves always contains $2n - 1$ nodes.
- In strictly binary tree, every node should have exactly two children or none.



Full Binary tree:

A full binary tree of height h has all its leaves at level h . alternatively; All non leaf nodes of a full binary tree have two children, and the leaf nodes have no children.

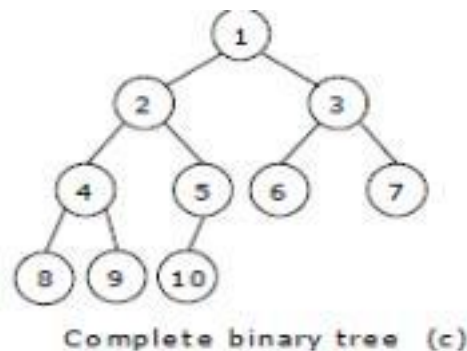
A full binary tree with height h has $2^{h+1} - 1$ nodes. A full binary tree of height h is a *strictly binary tree*.



Complete Binary tree:

A binary tree with n nodes is said to be **complete** if it contains all the first n nodes of the above numbering scheme. Figure 7.2.4 shows examples of complete and incomplete binary trees.

A complete binary tree of height h looks like a full binary tree down to level $h-1$, and the level h is filled from left to right.

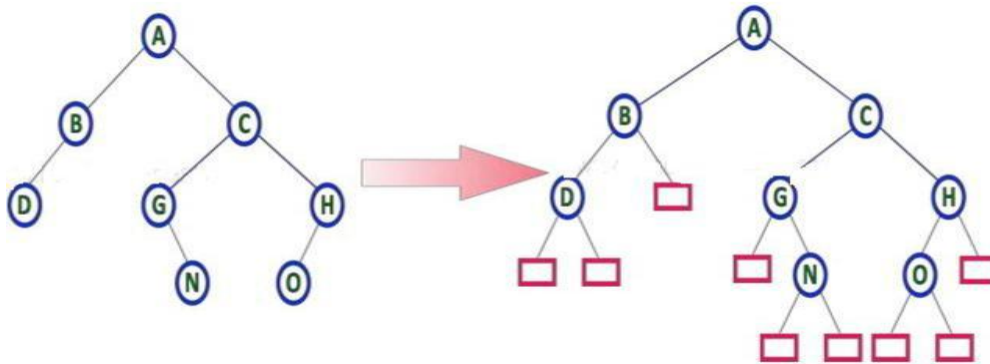


EXTENDED BINARY TREE :

- A binary tree can be converted into Full Binary tree by adding dummy nodes to existing nodes wherever required.

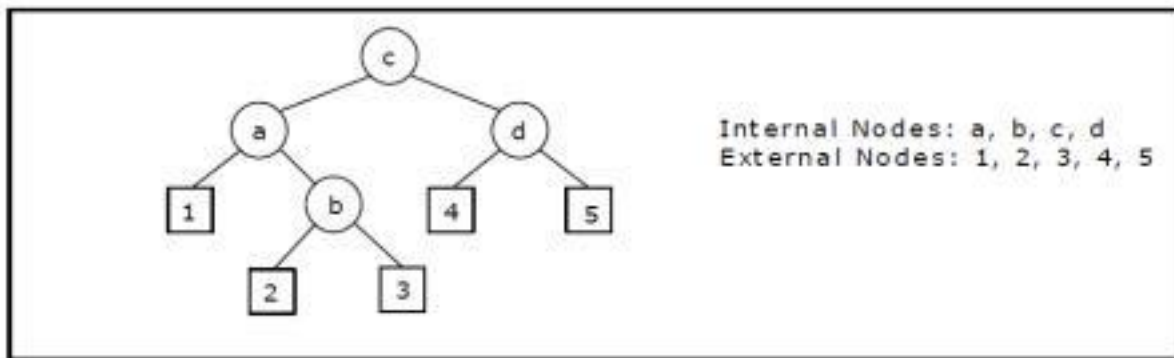


The full binary tree obtained by adding dummy nodes to a binary tree is called as Extended Binary Tree



Internal and external nodes:

We define two terms: Internal nodes and external nodes. An internal node is a tree node having at least one—key and possibly same children. It is same times convenient to have another types of nodes, called an external node, and pretend that all null child links point to such a node. An external node doesn't exist, but serves as a conceptual place holder for nodes to be inserted.



Ancestor and Descendent If there is a path from node A to node B, then A is called an ancestor of B and B is called a descendent of A.

Binary Tree Traversals:

Traversal of a tree means to visit each node in the tree exactly once. In a linear list nodes are visited from first to last, but a tree being a non linear , we need a definite rules. all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree. Tree can be traversed a number of ways. All of them differ only in the order in which they visit the nodes. The three main methods of traversing a tree are, Because, There are three ways which we use to traverse a tree –

- Inorder Traversal
- Preorder Traversal
- Postorder

Traversal In all of them we do not require to do anything to traverse an empty tree. All the traversal methods are based functions since a binary tree is itself recursive as every child of a node in a binary tree is itself a binary tree.

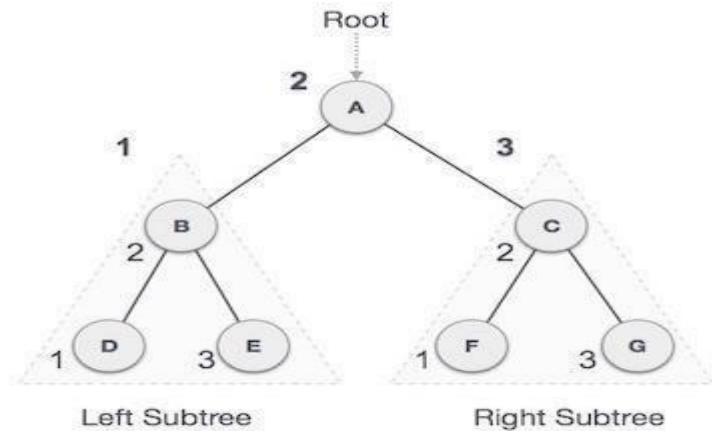
In-order Traversal

In this traversal method, the left subtree is visited first, then the root and later the right sub-tree. We should always remember that every node may represent a subtree itself.

If a binary tree is traversed **in-order**, the output will produce sorted key values in an ascending order. Algorithm

Until all nodes of the tree are not visited

1. Step 1 - Traverse the left subtree recursively.
2. Step 2 - Visit the root node.
3. Step 3 - Traverse the right subtree recursively.



We start from **A**, and following in-order traversal, we move to its left subtree **B**. **B** is also traversed in-order. The process goes on until all the nodes are visited. The output of in-order traversal of this tree will be –

D→B→E→A→F→C→G

Algorithm for in-order traversal :

Inorder (INFO, LEFT, RIGHT, ROOT)

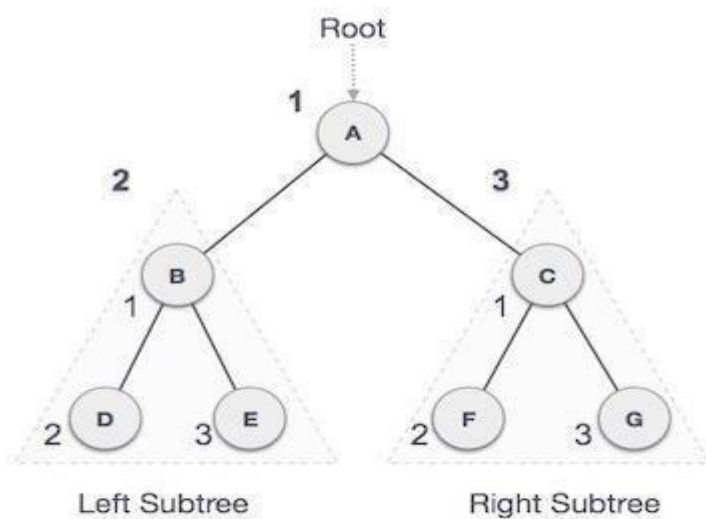
1. [Push NULL onto STACK and initialize PTR] Set TOP = 1, STACK[1] = NULL and PTR = ROOT
2. Repeat while PTR ≠ NULL [Push leftmost path onto STACK]
 - a. Set TOP = TOP + 1 and STACK [TOP] = PTR
 - b. Set PTR = LEFT [PTR]
3. Set PTR = STACK[TOP] and TOP = TOP – 1
4. Repeat steps 5 to 7 while PTR ≠ NULL
5. Apply process to INFO[PTR]
6. [Right Child?] If RIGHT [PTR] ≠ NULL Then
 - a. Set PTR = RIGHT [PTR]
 - b. goto step 2
7. Set PTR = STACK[TOP] and TOP = TOP – 1
End of Step 4 Loop
8. Exit

Pre-order Traversal

In this traversal method, the root node is visited first, then the left subtree and finally the right subtree. Algorithm- Until all nodes are traversed –

1. Visit the root node
2. Traverse the left subtree recursively.

3. Traverse the right subtree recursively.



We start from **A**, and following pre-order traversal, we first visit **A** itself and then move to its left subtree **B**. **B** is also traversed pre-order. The process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

A→B→D→E→C→F→G

Algorithm for preorder traversal :

Preorder (INFO, LEFT, RIGHT, ROOT)

1. [Initially push NULL onto STACK, and initialize PTR] Set TOP = 1,
STACK [1] = NULL
and PTR = ROOT
 2. Repeat steps 3 to 5 while PTR ≠ NULL
 3. Apply process to INFO [PTR]
 4. [Right child?] If RIGHT [PTR] ≠ NULL Then [Push on STACK]
 5. [Left child?] If LEFT [PTR] ≠ NULL
 then set PTR = LEFT[PTR]
 Else [Pop from STACK]
 set PTR = STACK[TOP] and TOP = TOP – 1
 Endif
- End of step 2
6. Exit

Post-order Traversal

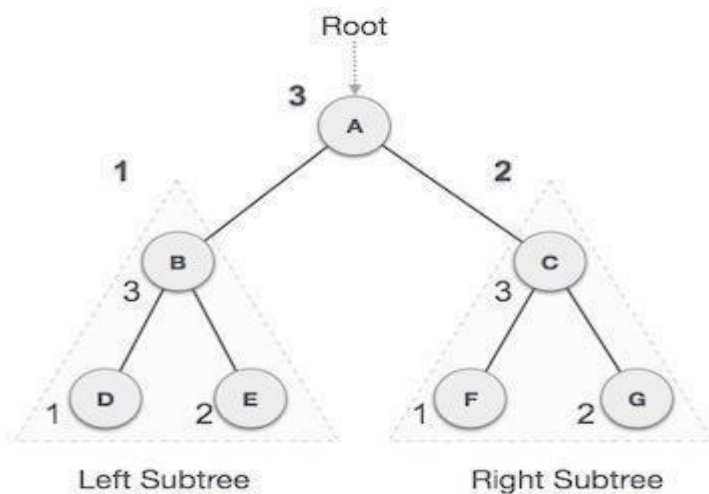
In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.

Algorithm

Until all nodes of the tree are not visited

1. Step 1 - Traverse the left subtree recursively.

2. Step 2 - Traverse the right subtree recursively.
3. Step 3 - Visit the root node.



We start from **A**, and following pre-order traversal, we first visit the left subtree **B**. **B** is also traversed post-order. The process goes on until all the nodes are visited. The output of post-order traversal of this tree will be

D→E→B→F→G→C→A

Algorithm for postorder traversal :

Postorder (INFO, LEFT, RIGHT, ROOT)

1. [Push NULL onto STACK and initialize PTR] Set TOP = 1, STACK[1] = NULL and PTR = ROOT
2. [Push leftmost path onto STACK] Repeat steps 3 to 5 while PTR ≠ NULL
3. Set TOP = TOP + 1 and STACK [TOP] = PTR [Pushes PTR on STACK]
4. If RIGHT [PTR] ≠ NULL Then
 - Set TOP = TOP + 1
 - and STACK [TOP] = – RIGHT [PTR]
 Endif
5. Set PTR = LEFT [PTR]
- End of step 2 loop
6. Set PTR = STACK [TOP] and TOP = TOP – 1 [Pops node from STACK]
7. Repeat while PTR > 0
 - a. Apply process to INFO [PTR]
 - b. Set PTR = STACK [TOP] and TOP = TOP – 1
 End loop
8. If PTR < 0 Then
 - a. Set PTR = – PTR
 - b. goto step 2
 Endif
9. Exit

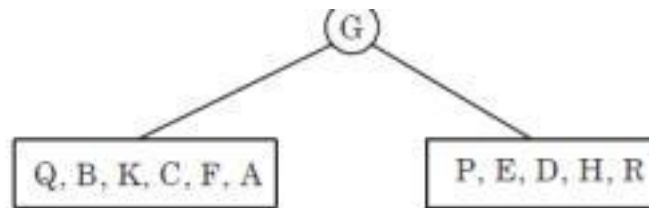
Question:. Construct a binary tree for the following :

Inorder : Q, B, K, C, F, A, G, P, E, D, H, R
Preorder : G, B, Q, A, C, K, F, P, D, E, R, H
Find the postorder of the tree.

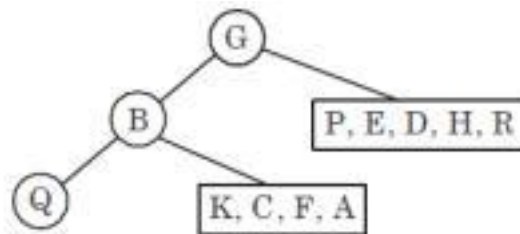
Answer Step 1 : In preorder traversal root is the first node. So, G is the root node of the binary tree. So, G root

(G)

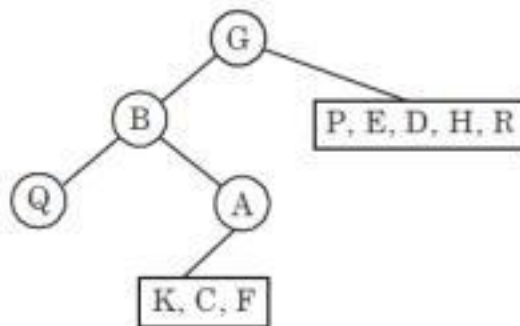
Step 2 : We can find the node of left sub-tree and right sub-tree with inorder sequence. So,



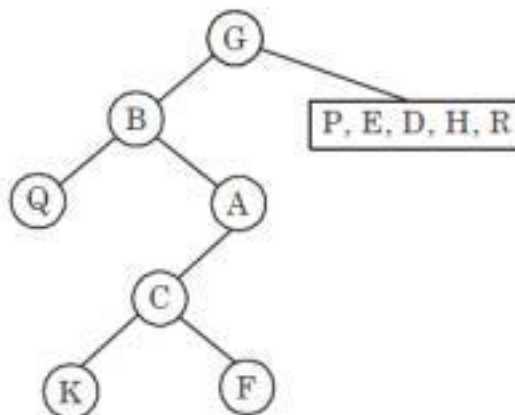
Step 3 : Now, the left child of the root node will be the first node in the preorder sequence after root node G. So



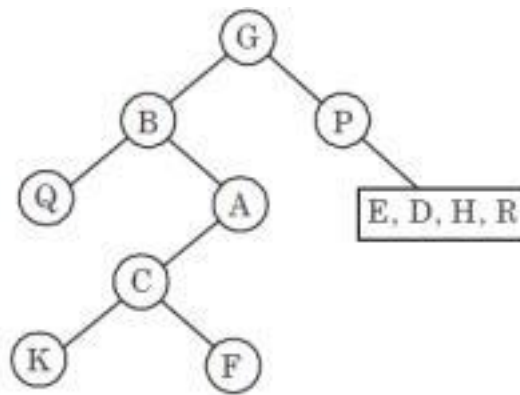
Step 4 : In inorder sequence, Q is on the left side of B and A is on the right side B. So,



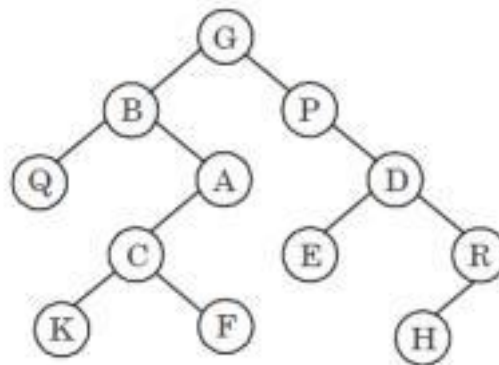
Step 5 : In inorder sequence, C is on the left side of A . Now according to inorder sequence, K is on the left side of C and F is on the right side of C.



Step 6 : Similarly, we can go further for right side of G



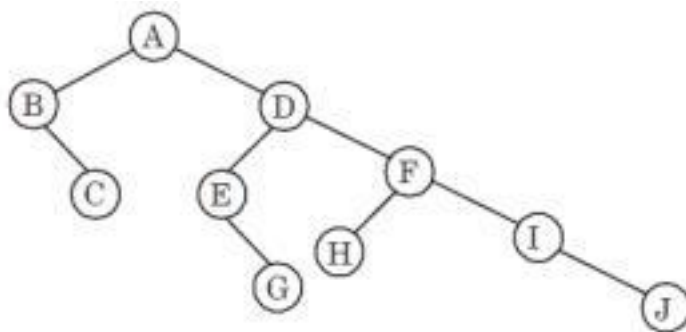
So, the final tree is



Postorder of tree : Q, K, F, A, B, E, H, R, D, P, G

Q.

Inorder : B C A E G D H F I J
Preorder : A B C D E F G H I J



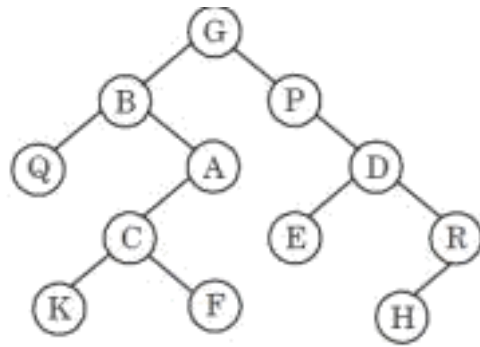
1. The order of nodes of a binary tree in inorder and postorder traversal are as follows:

In order : B, I, D, A, C, G, E, H, F.

Post order: I, D, B, G, C, H, F, E, A.

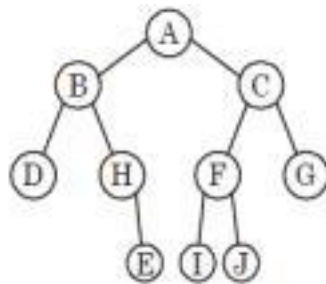
- (i) Draw the corresponding binary tree.
- (ii) Write the pre order traversal of the same tree.

2. Construct a binary tree for the following : Inorder : Q, B, K, C, F, A, G, P, E, D, H, R Preorder : G, B, Q, A, C, K, F, P, D, E, R, H Find the postorder of the tree.



Postorder of tree: Q,K,F,A,B,E,H,R,D,P,G

3. Draw a binary tree with following traversal : Inorder : *D B H E A I F J C G*
 Preorder : *A B D E H C F I J G* AKTU 2015



Program for Traversal:

```

#include<stdio.h>
#include<stdlib.h>
struct node
{
int value;
struct node* left;
struct node* right;
};
struct node* root;
struct node* insert(struct node* r, int data)
{
if(r==NULL)
{
r = (struct node*) malloc(sizeof(struct node));
r->value = data;
r->left = NULL;
r->right = NULL;
}
else if(data < r->value){
r->left = insert(r->left, data);
}
else {
r->right = insert(r->right, data);
}
return r;
}
void inorder(struct node* r)
{

```

```

if(r!=NULL){
inorder(r->left);
printf("%d ", r->value);
inorder(r->right);
}}
void preorder(struct node* r)
{
if(r!=NULL){
printf("%d ", r->value);
preorder(r->left);
preorder(r->right);
}} void postorder(struct node* r)
{
if(r!=NULL){
postorder(r->left);
postorder(r->right);
printf("%d ", r->value);
}}
int main()
{
root = NULL;
int n, v;
printf("How many data do you want to insert
?\n"); scanf("%d", &n);
for(int i=0; i<n; i++){
printf("Data %d: ",
i+1); scanf("%d", &v);
root = insert(root, v);
}
printf("Inorder Traversal :");
inorder(root); printf("\n");
printf("Preorder Traversal :");
preorder(root); printf("\n");
printf("Postorder Traversal
:"); postorder(root);
printf("\n");

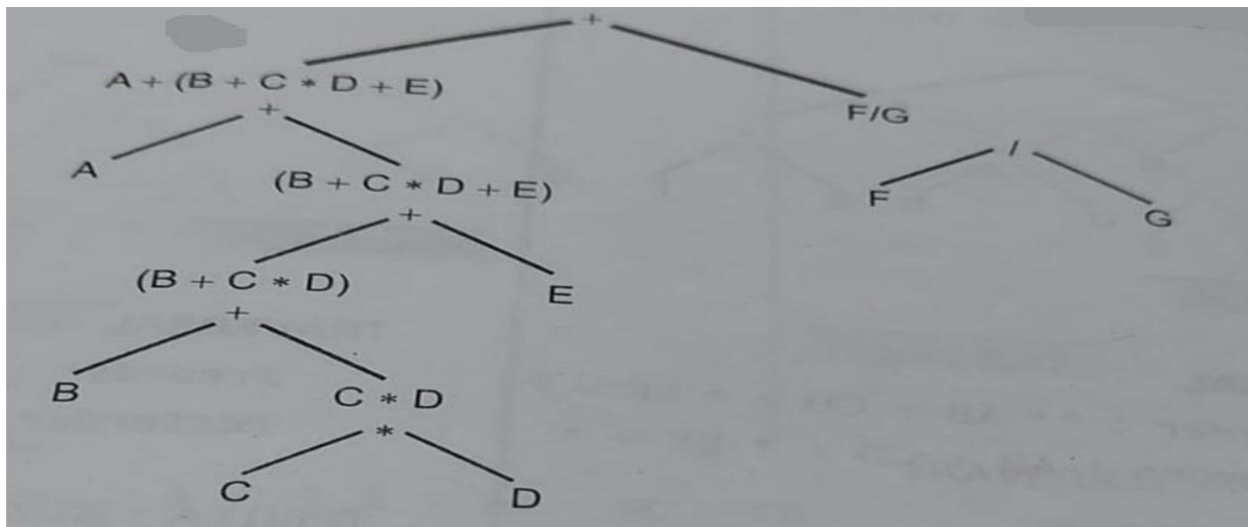
return 0;
}

```

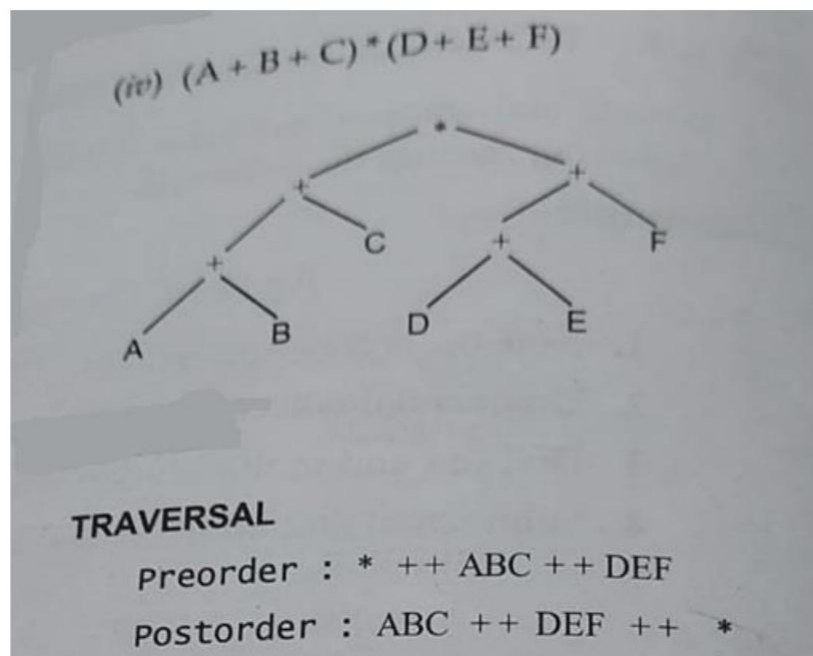
Technique of converting an expression into binary tree:

Divide and conquer technique is used to convert an expression into a binary. The following clarifies the technique. **Example: $A+(B+C*d+E)+F/G$**

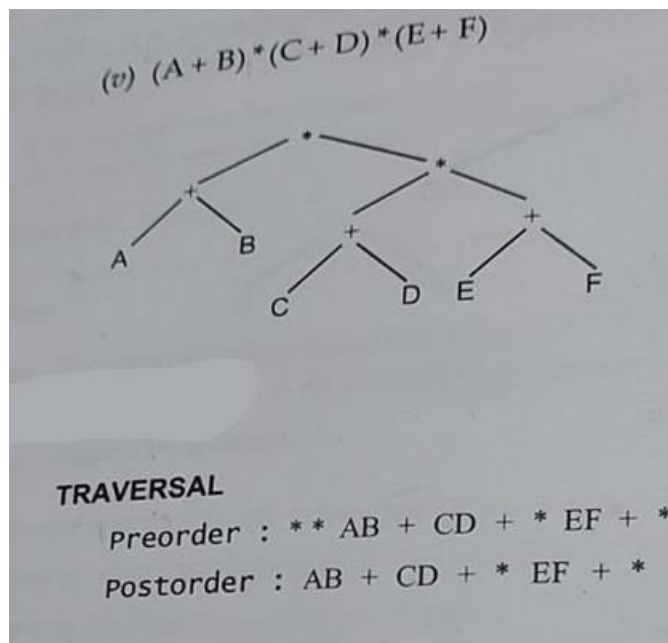
1. Note the order of precedence. All expressions in parentheses are to be evaluated first.
2. Exponential will come first.
3. Division and multiplication will be the next in order of precedence.
4. Subtraction and addition will be the last to be processed.



Example -2



Example 3:

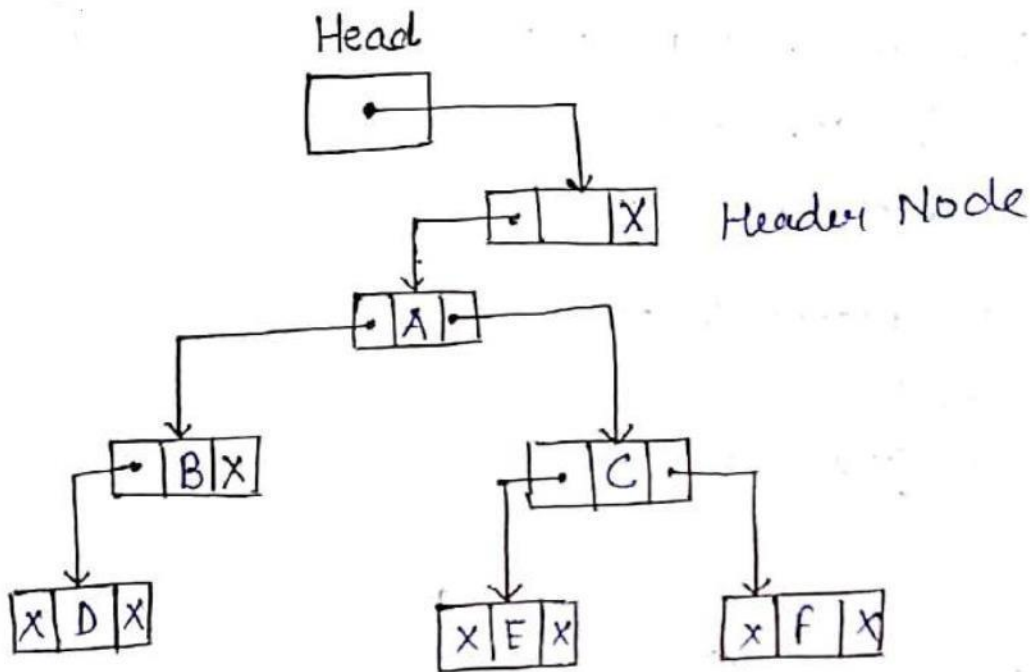


Threaded Binary tree:

Header Node: Sometimes an extra special node called a header node is added to the beginning of tree. When this extra node is used, then. The tree pointer variable head will point to the header node and the left pointer of the header node will point the root of the tree.

In the link list representation approximately half of the entries in the pointer field left and right will contain null elements.

We will replace certain null entries by special pointer, which points to nodes higher in the tree. These special pointers are called threads, I had a binary tree. With such pointers are called threaded binary tree.



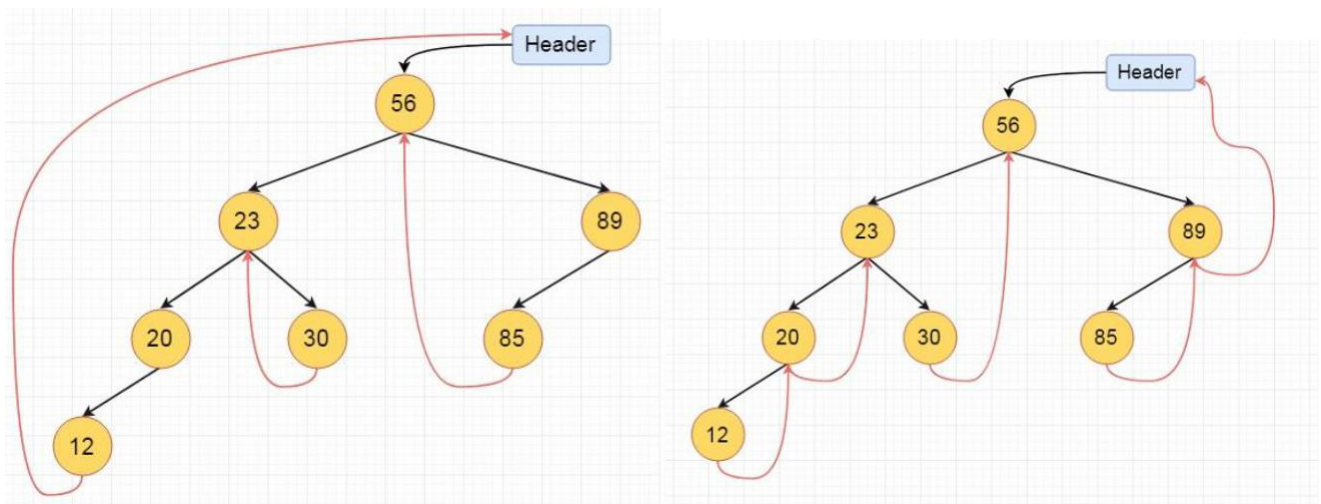
The binary tree may have at most two children. But if they have only one children, or no children, the link part in the linked list representation remains null. Using threaded binary tree representation, we can reuse that empty links by making some threads.

If one node has some vacant left or right child area, that will be used as thread. There are two types of threaded binary tree. The single threaded tree or fully threaded binary tree. In single threaded mode, there are another two variations. Left threaded and right threaded.

In the left threaded mode if some node has no left child, then the left pointer will point to its inorder predecessor, similarly in the right threaded mode if some node has no right child, then the right pointer will point to its inorder successor. In both cases, if no successor or predecessor is present, then it will point to header node.

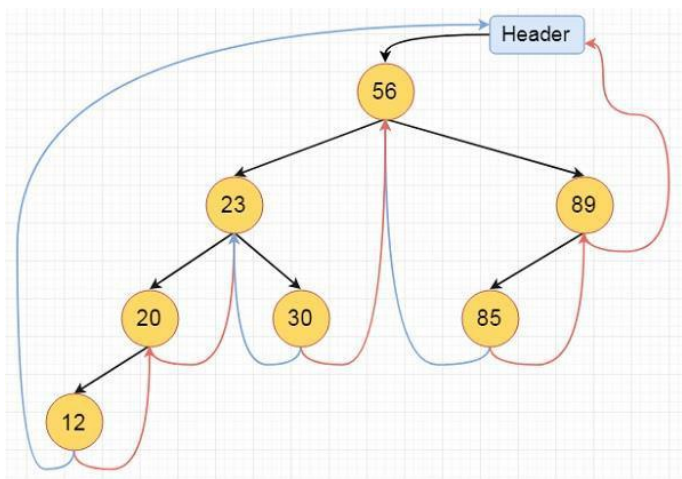
For fully threaded binary tree, each node has five fields. Three fields like normal binary tree node, another two fields to store Boolean value to denote whether link of that side is actual link or thread.

Left Thread Flag	Left Link	Data	Right Link	Right Thread Flag
------------------	-----------	------	------------	-------------------



These are the examples of left and right threaded tree

This is the fully threaded binary tree



Advantages of using threaded binary tree :

1. In threaded binary tree the traversal operations are very fast.
2. In threaded binary tree, we do not require stack to determine the predecessor and successor node.
3. In a threaded binary tree, one can move in any direction *i.e.*, upward or downward because nodes are circularly linked.
4. Insertion into and deletions from a threaded tree are all although time consuming operations but these are very easy to implement.

Binary Search Tree:

A binary search tree is a binary s tree. Which is either empty or satisfy the following rules:

- Every element has a key and no two elements have the same value.
- The values of the key in the left child or left subtree is less than the value of the root.
- The value of the key in the right child or right sub tree is more than or equal to the value of the root.
- The left and right subtree of the root are also a binary search tree.

Searching:

Search(info, left, right, root, item, loc, par)

a binary search tree t is in memory and an item of information is given. this procedure finds the location loc of item in t and also the location par of the parent of item. there are three special cases:

- (i) $loc = \text{null}$ and $par = \text{null}$ will indicate that the tree is empty.
 - (ii) $loc \neq \text{null}$ and $par = \text{null}$ will indicate that item is the root of t .
 - (iii) $loc = \text{null}$ and $par \neq \text{null}$ will indicate that item is not in t and can be added to t as a child of the node n with location par .
- 1 [tree empty?] if $\text{root} = \text{null}$, then: set $loc := \text{null}$ and $par := \text{null}$. and return
 - 2 [item at root?] if $\text{item} = \text{info}[\text{root}]$, then: set $loc := \text{root}$ and $par := \text{null}$. and return
 - 3 [initialize pointers ptr and $save$.]
if $\text{item} < \text{info}[\text{root}]$, then:
set $ptr = \text{left}[\text{root}]$ and $save = \text{root}$
else: $ptr = \text{right}[\text{root}]$ and $save = \text{root}$
(end of if)
 4. Repeat steps 5 and 6 while $ptr \neq \text{null}$
 5. [item found?] if $\text{item} = \text{info}[ptr]$ then:
set $loc := ptr$ and $par := save$, and return (end if)
 6. if $\text{item} < \text{info}[ptr]$ then:
set $save := ptr$ and $ptr := \text{left}[ptr]$
else: set $save := ptr$ and $ptr := \text{right}[ptr]$. [end of if structure.] (end of step 4 loop.)
 7. search unsuccessful set $loc := \text{null}$ and $par := save$.
 8. exit

Complexity of the Searching Algorithm. = $O(\log_2 n)$

Search(x, item)

(x is a root and item is searched element)

- 1 if $x = \text{null}$ or $\text{item} = \text{key}[x]$
Then return x
- 2 if $\text{item} < \text{key}[x]$
Then return $\text{Search}(\text{left}[x], \text{item})$
Else return $\text{Search}(\text{right}[x], \text{item})$
3. exit
4. **Complexity of the Searching Algorithm. = $O(h)$**

Insertion:

Insbst(info, left, right, root, avail, item, loc)

A binary search tree t is in memory and an item of information is given. this algorithm finds the location loc of item in t or adds item as a new node in t at location loc .

1. call $\text{find}(\text{info}, \text{left}, \text{right}, \text{root}, \text{item}, \text{loc}, \text{par})$.
2. if $loc \neq \text{null}$, then exit.
3. (copy item into new node in avail list.)
 - (a) if $\text{avail} = \text{null}$, then: write: overflow. and exit.
 - (b) set $\text{new} := \text{avail}$. $\text{avail} := \text{left}[\text{avail}]$ and $\text{info}[\text{new}] = \text{item}$.
 - (c) set $loc := \text{new}$, $\text{left}[\text{new}] = \text{null}$ and $\text{right}[\text{new}] = \text{null}$.
4. [add item to tree.] if $par = \text{null}$, then:
set $\text{root} := \text{new}$.
else if $\text{item} < \text{info}[par]$, then:


```

    set left[par] = new.
    else: set right[par] = new.
  [end of if structure.]
5. exit

```

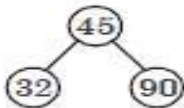
Example: Make a binary search tree for the following sequence of numbers, show all steps : 45, 32, 90, 34, 68, 72, 15, 24, 30, 66, 11, 50, 10.

Numerical :

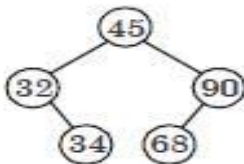
1. **Insert 45 :**



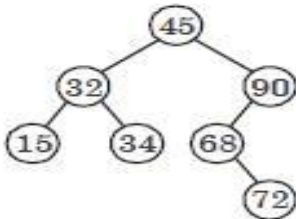
3. **Insert 90 :**



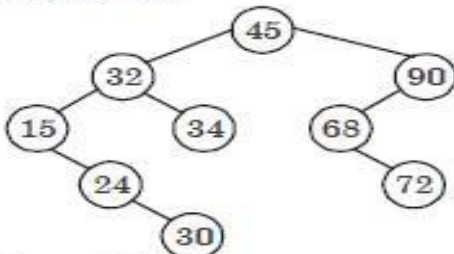
5. **Insert 68 :**



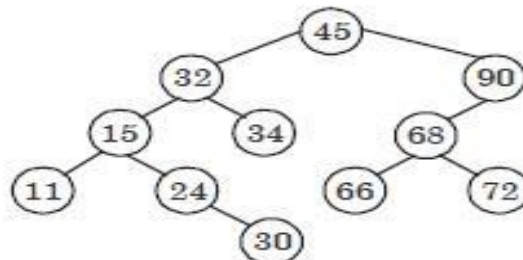
7. **Insert 15 :**



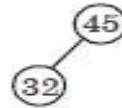
9. **Insert 30 :**



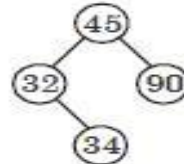
11. **Insert 11 :**



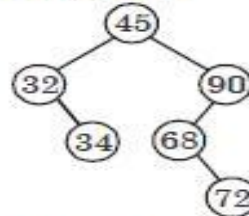
2. **Insert 32 :**



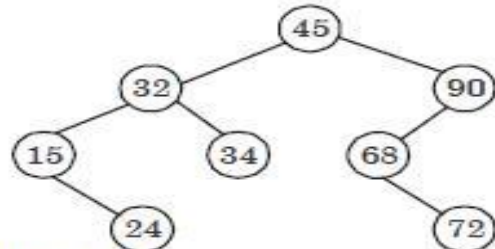
4. **Insert 34 :**



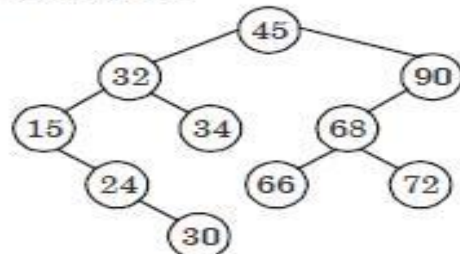
6. **Insert 72 :**



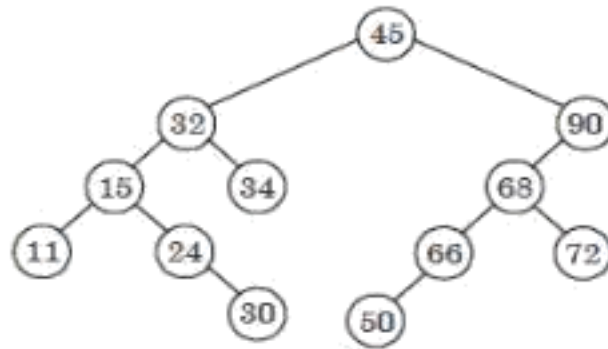
8. **Insert 24 :**



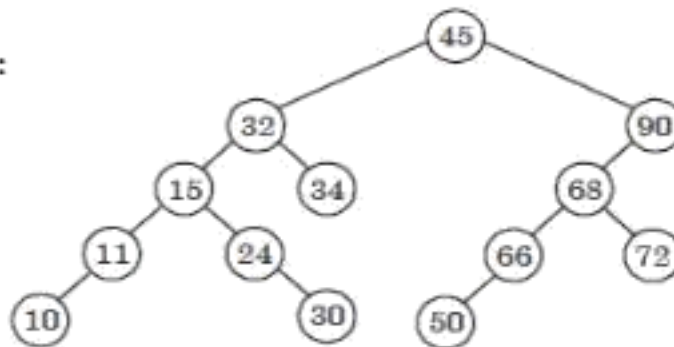
10. **Insert 66 :**



12. Insert 50 :



13. Insert 10 :



Deletion:

Suppose T is a binary search tree, and suppose an ITEM of information is given. This section gives an algorithm which deletes ITEM from the tree T.

The deletion algorithm first uses search algorithm to find the location of the node N which contains ITEM and also the location of the parent node P(N). The way N is deleted from the tree depends primarily on the number of children of node N. There are three cases:

Case 1. N has no children. Then N is deleted from T by simply replacing the location of N in the parent node P(N) by the null pointer.

Case 2. N has exactly one child. Then N is deleted from T by simply replacing the location of N in P(N) by the location of the only child of N.

Case 3. N has two children. Let S(N) denote the inorder Successor of N. (The reader can verify that S(N) does not have a left child.) Then N is deleted from T by first deleting S(N) from T (by using Case 1 or Case 2) and then replacing node N in T by the node S(N).

Observe that the third case is much more complicated than the first two cases. In all three cases, the memory space of the deleted node N is returned to the AVAIL list.

CASEA(info, left, right, root, loc, par)

this procedure deletes the node n at location loc, where n does not have two children. the pointer par gives the location of the parent of n, or else par = null indicates that n is the root node. the pointer child gives the location of the only child of n, or else child = null indicates n has no children.

1. [initializes child.]
if left[loc] = null and right[loc] = null, then:
 set child:= null.
else if left[loc] ≠ null, then:
 set child:= left[loc]
else set child:=right[loc]
[end of if structure.]

2. if $par \neq null$, then:
 - if $loc = left[par]$, then:
 - set $left[par] = child$.
 - else: set $right[par] = child$.
 [end of if structure.]
 - else: set $root := child$.
 [end of if structure]
3. return.

CASEB(info, left, right, root, loc, par)

This procedure will delete the node n at location loc , where n has two children. the pointer par gives the location of the parent of n , or else $par = null$ indicates that n is the root node. the pointer suc gives the location of the inorder successor of n , and $parsuc$ gives the location of the parent of the inorder successor.

1. [find suc and $parsuc$.]
 - (a) set $ptr = right[loc]$ and save $:= loc$.
 - (b) repeat while $left[ptr] \neq null$
 - set $save = ptr$ and $ptr = left[ptr]$
 [end of loop.]
 - (c) set $suc = ptr$ and $parsuc := save$.
2. [delete inorder successor, using procedure CASEA]
 - call CASEA(info, left, right, root, suc, parsuc).
3. [replace node n by its inorder successor.]
 - (a) if $par \neq null$, then:
 - if $loc = left[par]$, then:
 - set $left[par] = suc$.
 - Else if : set $right[par] = suc$.
 [end of if structure.]
 - Else: set $root := suc$.
 [end of if structure]
 - (b) set $left[suc] = left[loc]$ and
 $Right[suc] = right[loc]$
4. return

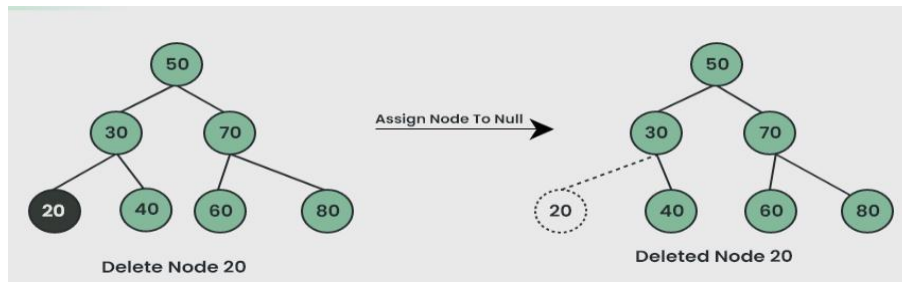
DEL(INFO, LEFT, RIGHT, ROOT, AVAIL, ITEM)

A binary search tree t is in memory, and an item of information is given. this algorithm deletes item from the tree.

1. [find the locations of item and its parent, using search algorithm] call Search(info, left, right, root, item, loc, par).
2. [item in tree?] if $loc = null$, then: write: item not in tree, and exit.
3. [delete node containing item,]
 - if $right[loc] \neq null$. and $left[loc] \neq null$, then:
 - call CASEB(info, left, right, root, loc, par).
 - else:
 - call CASEA(info, left, right, root, loc, par).
 (end of if structure.)
- 4.[return deleted node to the avail list.]
 - set $left[loc] = avail$ and $avail := loc$.
5. exit.

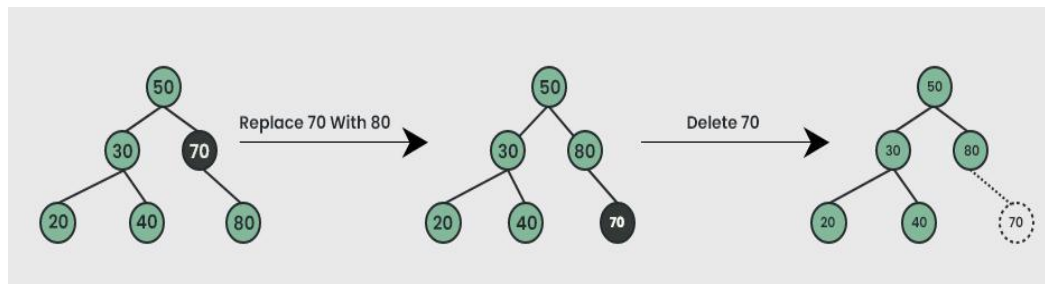
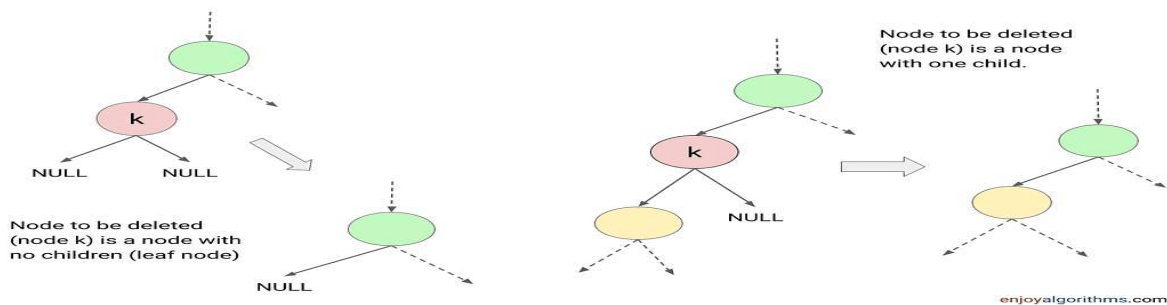
Deletion:

Based on the above idea, the strategy for deleting a node from a binary search tree has three cases. **Case 1:** Node to be deleted is a node with no children (leaf node) We just delete that node from BST.



Case 2: Node to be deleted is a node with one child

There is one parent and one grandchild, so we can link the grandchild directly to the parent node and delete that node.



Case 3: Node to be deleted is a node with two children

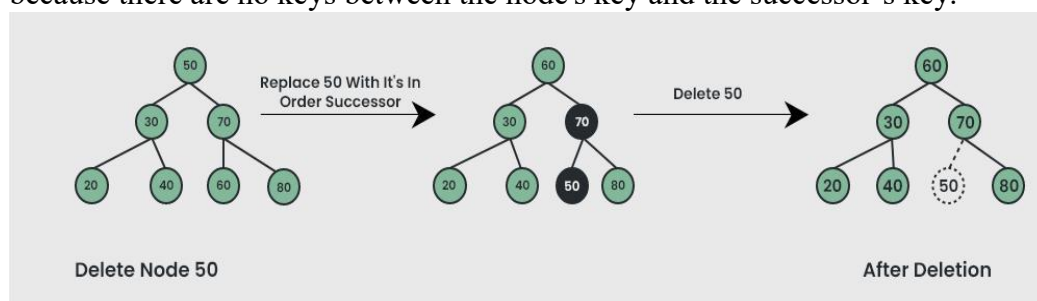
This one is a little bit tricky because we have to follow three key steps:

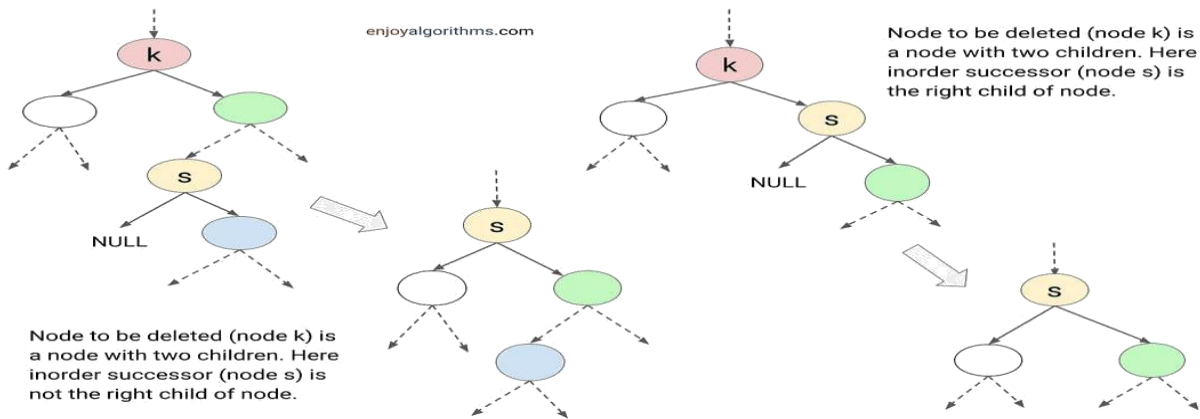
We need to first identify the in-order successor of that node, i.e., a node with a key just greater than the key of the given node to be deleted.

Then we need to replace the key of the given node with the key of the in-order successor.

Delete the in-order successor.

Because that node has a right child, its in-order successor is the node with the smallest key in its right subtree (leftmost descendant in the right subtree). So this replacement will preserve the BST property because there are no keys between the node's key and the successor's key.





```
#include<stdio.h>
#include<stdlib.h>
```

```
struct BST
{
int data;
struct BST *left;
struct BST *right;
};

struct BST *root= NULL;
struct BST *create()
{
struct BST *temp;
printf("\nEnter data:");
temp=(struct BST*)malloc(sizeof(struct BST));
scanf("%d",&temp->data);
temp->left=temp->right=NULL;
return temp;
}

void insert(struct BST *root,struct BST *temp)
{
{
if(temp->data<root->data)
{
if(root->left!=NULL)
insert(root->left,temp);
else
root->left=temp;
}
if(temp->data>root->data)
{
if(root->right!=NULL)
insert(root->right,temp);
else
root->right=temp;
}
}
}

void inorder(struct BST *root)
```

```

{
if(root!=NULL)
{
    inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
}
}

```

```

struct BST *find_minimum(struct BST *root)
{
    if(root == NULL)
        return NULL;
    else if(root->left != NULL) // node with minimum value will have no left child
        return find_minimum(root->left); // left most element will be minimum
    return root;
}

```

```

struct BST *delete(struct BST *root, int x)
{
    //searching for the item to be deleted
    if(root==NULL)
        return NULL;
    if (x>root->data)
        root->right = delete(root->right, x);
    else if(x<root->data)
        root->left = delete(root->left, x);
    else
    {
        //No Children
        if(root->left==NULL && root->right==NULL)
        {
            free(root);
            return NULL;
        }

        //One Child
        else if(root->left==NULL || root->right==NULL)
        {
            struct BST *temp;
            if(root->left==NULL)
                temp = root->right;
            else
                temp = root->left;
            free(root);
            return temp;
        }

        //Two Children
        else
        {
            struct BST *temp = find_minimum(root->right);

```

```

        root->data = temp->data;
        root->right = delete(root->right, temp->data);
    }
}
return root;
}

```

```

int main()
{
char ch;
int val;
struct BST *root=NULL,*temp;
do
{
temp=create();
if(root==NULL)
root=temp;
else
insert(root,temp);
printf("\nDo you want to enter more(y/n)?");
getchar();
scanf("%c",&ch);
}while(ch=='y'|ch=='Y');
printf("\nPreorder Traversal: ");
inorder(root);

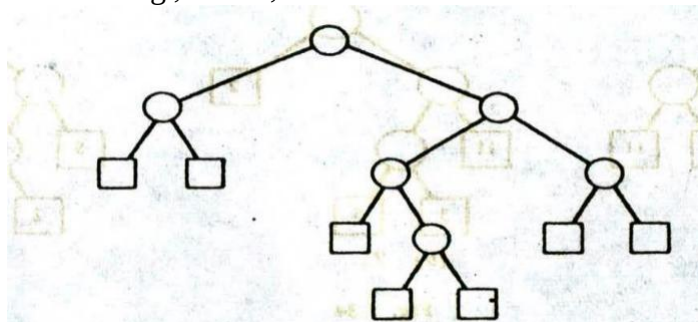
printf("\n\n");
printf("Enter which value you want to delete");
scanf("%d",&val);
root = delete(root, val);
    inorder(root);
return 0;
}

```

PATH LENGTHS; HUFFMAN'S ALGORITHM

An extended binary tree or 2-tree is a binary tree T in which each node has either 0 or 2 children. The nodes with 0 children are called external nodes, and the nodes with 2 children are called internal nodes. Figure shows a 2-tree where the internal nodes are denoted by circles and the external nodes are denoted by squares. In any 2-tree, the number N_E of external nodes is 1 more than the number N_I of internal nodes; that is, $N_E = N_I + 1$.

For example, for the 2-tree in Fig., $N_I = 6$, and $N_E = N_I + 1 = 7$.



Frequently, an algorithm can be represented by a 2-tree T where the internal nodes represent tests and the external nodes represent actions. We define the external path length L_E of a 2-tree T to be the sum of all path lengths summed over each path from the root R of T to an external node. The internal path length L_I of T is defined analogously, using internal nodes instead of external nodes.

$L_E = 2+2+3+4+4+3+3=21$ and $L_I = 0+1+1+2+3+2=9$

Observe that $L_I + 2n = 9 + 2*6 = 9 + 12 = 21 = L_E$

where $n = 6$ is the number of internal nodes. in fact, the formula $L_E = L_I + 2n$ is true for any 2-tree with n internal nodes.

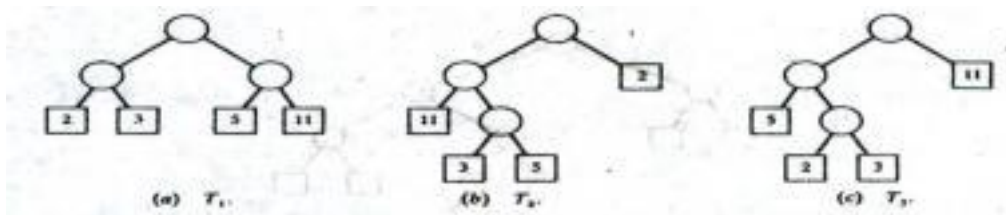
EXAMPLE : Figure shows three 2-trees, T_1 , T_2 , and T_3 , each having external nodes with weights 2, 3, 5 and 11. The weighted, path lengths of the three trees we as follows:

$$P_1 = 2*2 + 3*2 + 5*2 + 11*2 = 42$$

$$P_2 = 2*1 + 3*3 + 5*3 + 11*2 = 48$$

$$P_3 = 2*3 + 3*3 + 5*2 + 11*1 = 36$$

The quantities P_1 and P_3 indicate that the complete tree need not give a minimum length P , and the quantities P_2 and P_3 indicate that similar trees need not give the same lengths.

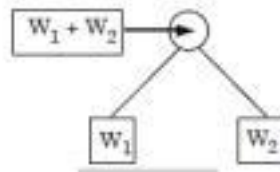


Huffman algorithm

Huffman tree is a binary tree in which each node in the tree represents a symbol and each leaf represent a symbol of original alphabet.

Huffman algorithm :

1. Suppose, there are n weights $W_1, W_2, \dots, W_n$.
2. Take two minimum weights among the n given weights. Suppose W_1 and W_2 are first two minimum weights then subtree will be :



3. Now the remaining weights will be $W_1 + W_2, W_3, W_4, \dots, W_n$.
4. Create all subtree at the last weight.

Example: Create a Huffman tree and Huffman code for each five different symbols. The symbol's frequencies are:

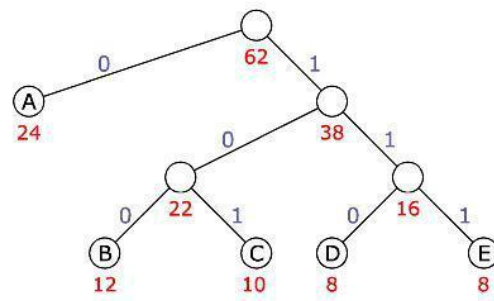
Symbol Frequency

A	24
B	12
C	10
D	8
E	8

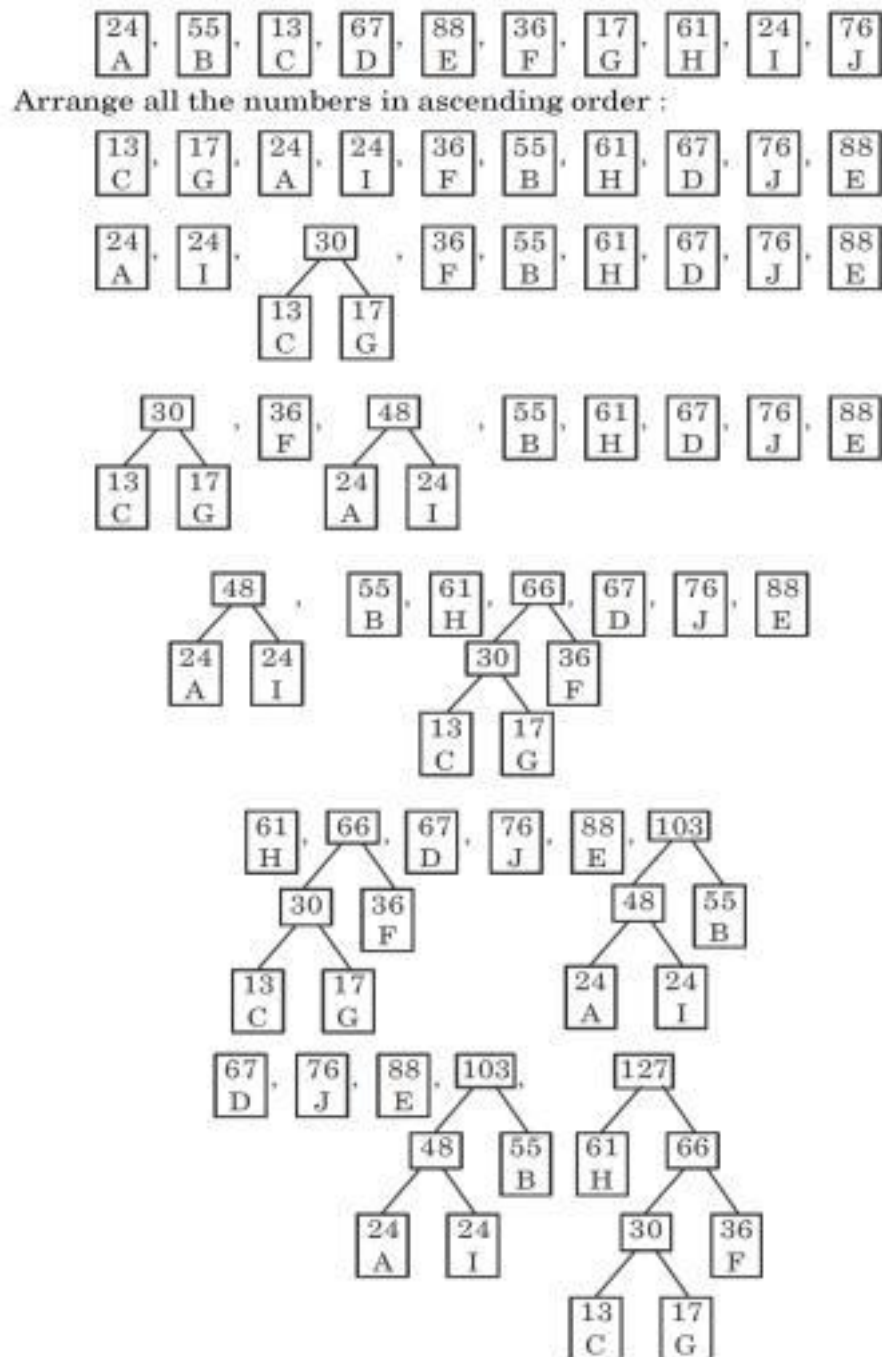
(with 3 bit per code word)

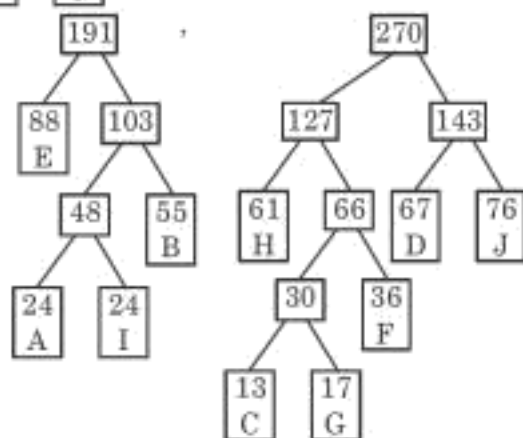
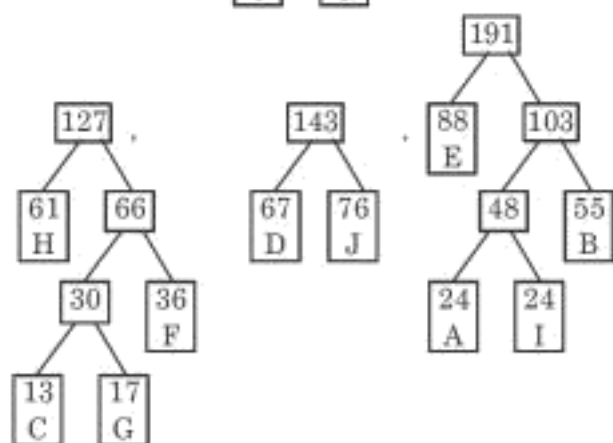
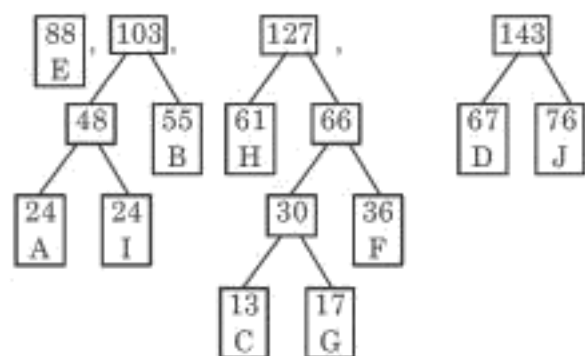
The two rarest symbols 'E' and 'D' are connected first, followed by 'C' and 'D'. The new parent nodes have the frequency 16 and 22 respectively and are brought together in the next step. The resulting node and the remaining symbol 'A' are subordinated to the root node that is created in a final step.

Code Tree according to Huffman

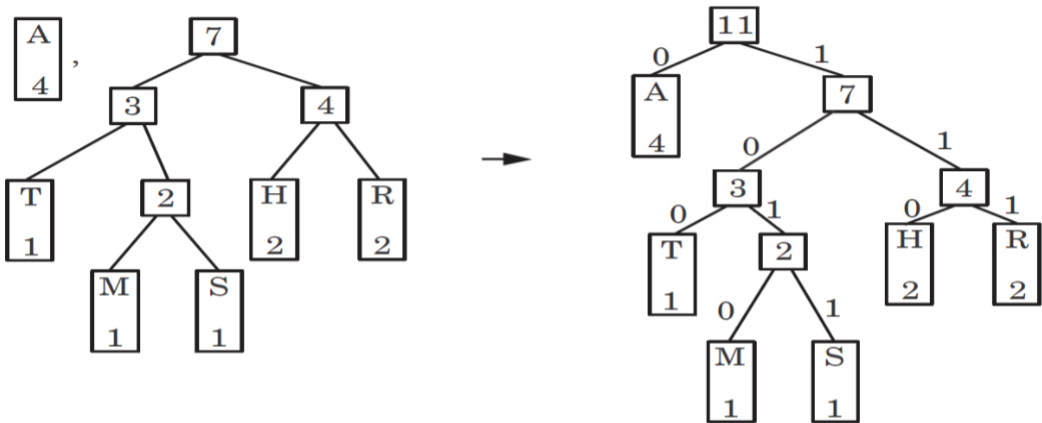
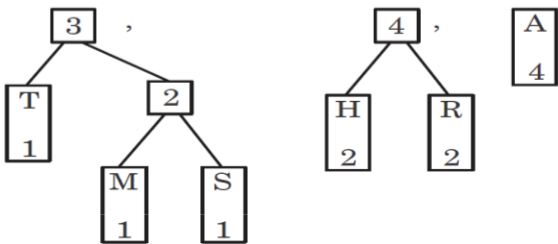
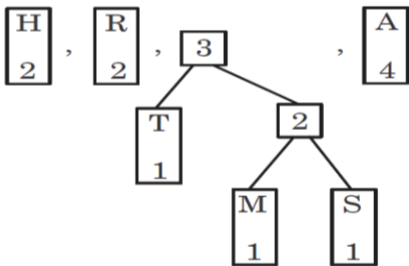
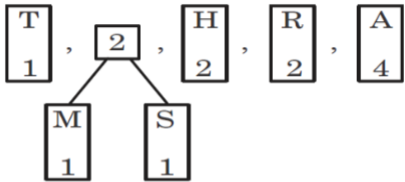
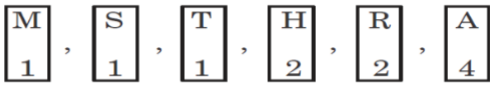
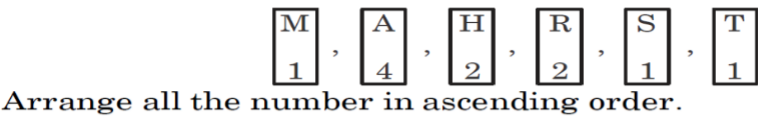


Example: Create a Huffman tree with following numbers : 24, 55, 13, 67, 88, 36, 17, 61, 24, 76





Q. Construct Huffman tree for *MAHARASHTRA* with its optimal code.



Character	Code
M	1010
A	0
H	110
R	111
S	1011
T	100

AVL Tree

An **AVL tree** (Adelson-Velskii and Landis' tree, named after the inventors) is a self-balancing binary search tree. In an AVL tree(or height balanced tree), the heights of the two child subtrees of any node differ by at most one; if at any time they differ by more than one, rebalancing is done to restore this property.. Insertions and deletions may require the tree to be rebalanced by one or more tree rotations.

The balance factor is calculated as follows: $\text{balance factor} = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$. For each node checked, if the balance factor remains -1 , 0 , or $+1$ then no rotations are necessary. However, if balance factor becomes less than -1 or greater than $+1$, the subtree rooted at this node is unbalanced.

In other words,

An AVL tree is a binary search tree which has the following properties:

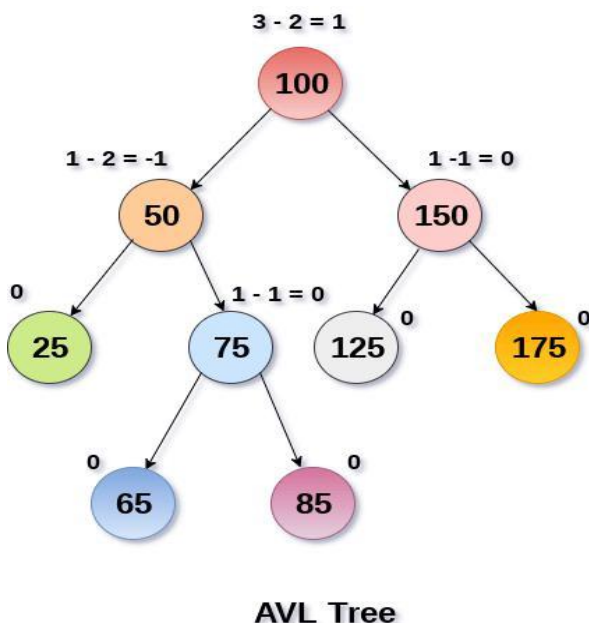
1. The sub-trees of every node differ in height by at most one.
2. Every sub-tree is an AVL tree.

If balance factor of any node is 1 , it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0 , it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1 , it means that the left sub-tree is one level lower than the right sub-tree.

An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and $+1$. therefore, it is an example of AVL tree.



Operations on AVL tree

AVL tree is also a binary search tree therefore, all the operations are performed in the same way as they are performed in a binary search tree. Searching and traversing do not lead to the violation in property of AVL tree. However, insertion and deletion are the operations which can violate this property and therefore, they need to be revisited.

1 Insertion: Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.

2 Deletion: Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

AVL Tree Rotations

We perform rotation in AVL tree only in case if Balance Factor is other than **-1, 0, and 1**. There are basically four types of rotations which are as follows:

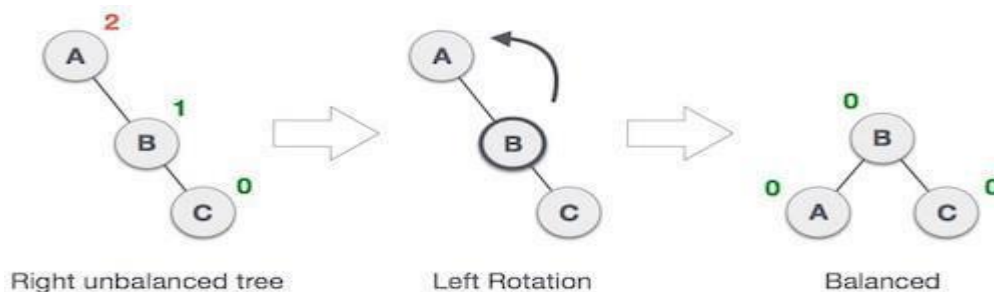
1. L L rotation: Inserted node is in the left subtree of left subtree of A
2. R R rotation : Inserted node is in the right subtree of right subtree of A
3. L R rotation : Inserted node is in the right subtree of left subtree of A
4. R L rotation : Inserted node is in the left subtree of right subtree of A

Where node A is the node whose balance Factor is other than -1, 0, 1.

The first two rotations LL and RR are single rotations and the next two rotations LR and RL are double rotations. For a tree to be unbalanced, minimum height must be at least 2, Let us understand each rotation

1. RR Rotation

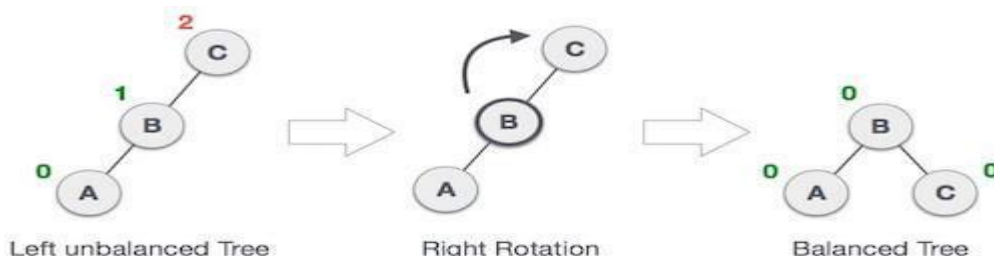
When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation, RR rotation is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2



In above example, node A has balance factor -2 because a node C is inserted in the right subtree of A right subtree. We perform the RR rotation on the edge below A.

2. LL Rotation

When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, LL rotation is clockwise rotation, which is applied on the edge below a node having balance factor 2.



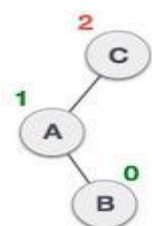
In above example, node C has balance factor 2 because a node A is inserted in the left subtree of C left subtree. We perform the LL rotation on the edge below A.

3. LR Rotation

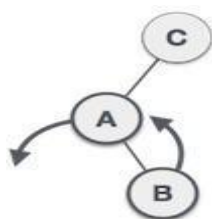
Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

AD

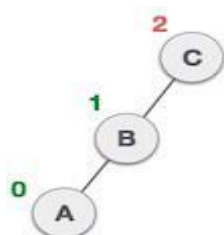
Example:



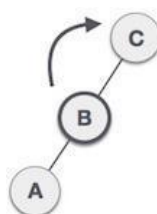
A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C



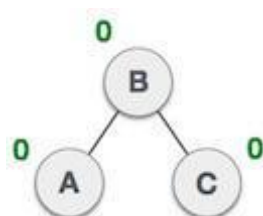
As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node A, has become the left subtree of B.



After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of C



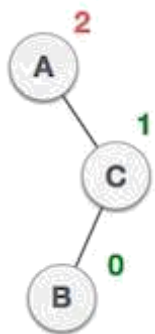
Now we perform LL clockwise rotation on full tree, i.e. on node C. node C has now become the right subtree of node B, A is left subtree of B



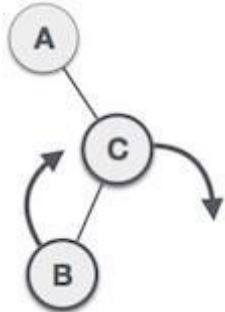
Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

4. RL Rotation

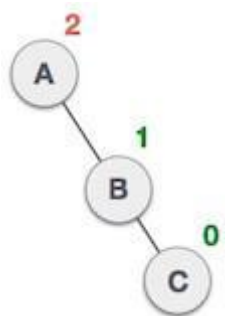
As already discussed, that double rotations are bit tougher than single rotation which has already explained above. R L rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.



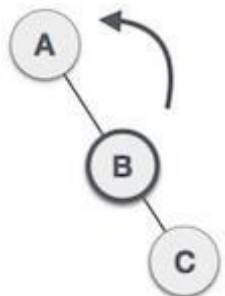
A node **B** has been inserted into the left subtree of **C** the right subtree of **A**, because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A



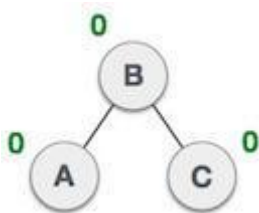
As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at **C** is performed first. By doing RR rotation, node **C** has become the right subtree of **B**.



After performing LL rotation, node **A** is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node A.



Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A. node **C** has now become the right subtree of node B, and node A has become the left subtree of B.



Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.

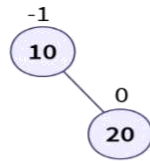
1. Insertion

In Binary Search Trees (BSTs), a new node (let's say N) is inserted as a leaf node by replacing the NULL value of a node's child. Similarly, in AVL Trees, the new node is also inserted as a leaf node, with its balance factor initially set to 0. However, after each insertion, the balance factors of the ancestors of the newly inserted node are checked to ensure tree balance. Only the ancestors are examined because the insertion only affects their heights, potentially inducing an imbalance. This process of traversing the ancestors to find the unbalanced node is called retracing.

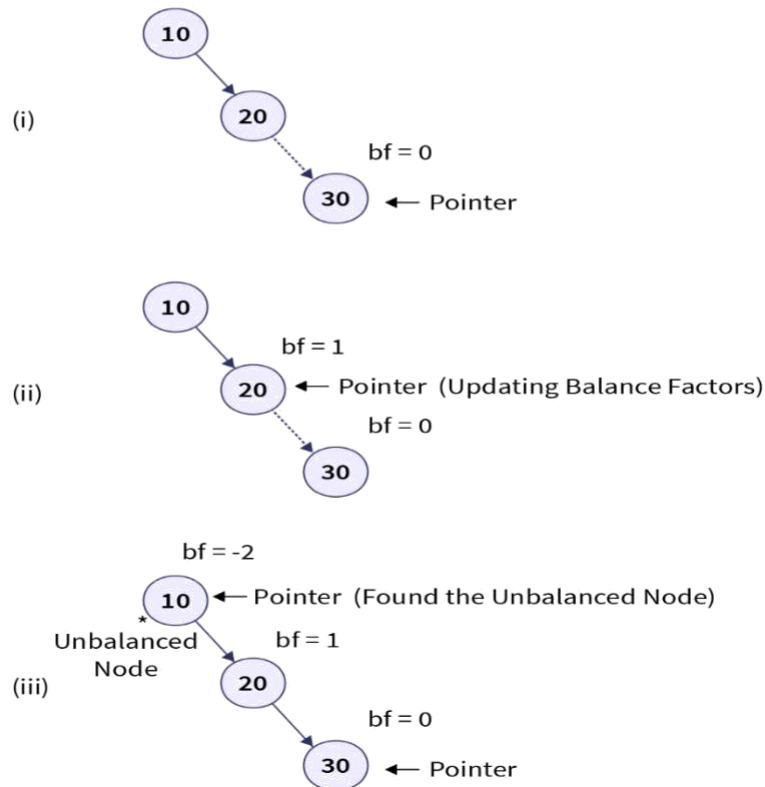
If the tree becomes unbalanced after inserting a new node, retracing helps us in finding the location of a node in the tree at which we need to perform the tree rotations to balance the tree.

The below gif demonstrates the retracing process upon inserting a new element in the AVL Tree:

- Example Tree



- Inserting a New Node and Retracing



2. Deletion

When an element is to be deleted from a Binary Search Tree, the tree is searched using various comparisons via the BST rule till the currently traversed node has the same value as that of the specified element. If the element is found in the tree, there are three different cases in which the deletion operation occurs depending upon whether the node to be deleted has any children or not: Case 1: When the node to be deleted is a leaf node

In this case, the node to be deleted contains no subtrees i.e., it's a leaf node. Hence, it can be directly removed from the tree.

Case 2: When the node to be deleted has one subtree

In this case, the node to be deleted is replaced by its only child thereby removing the specified node from the BST.

Case 3: When the node to be deleted has both the subtrees.

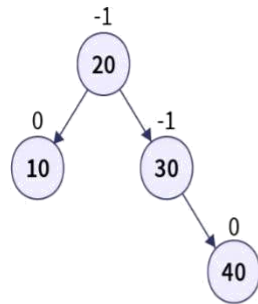
In this case, the node to be deleted can be replaced by one of the two available nodes:

It can be replaced by the node having the largest value in the left subtree (Longest left node or Predecessor). Or, it can be replaced by the node having the smallest value in the right subtree (Smallest right node or Successor).

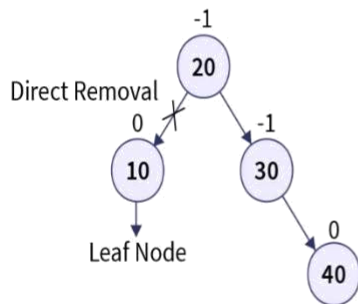
Just like the deletion operation in Binary Search Trees, the elements are deleted from AVL Trees depending upon whether the node has any children or not. However, upon every deletion in AVL Trees, the balance factor is checked to verify that the tree is balanced or not. If the tree becomes unbalanced after deletion, certain rotations are performed to balance the Tree.

→ Deletion

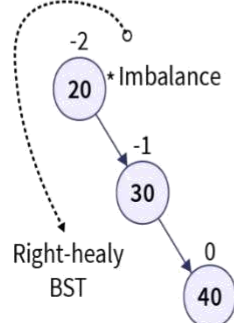
- Given AVL Tree



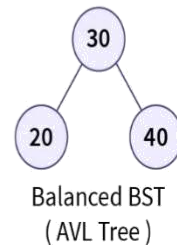
- Deleting the element 10



- Resulting Tree

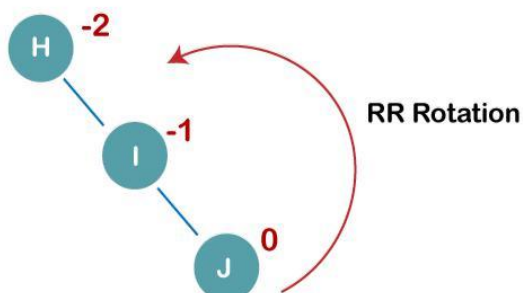


RR Rotation →

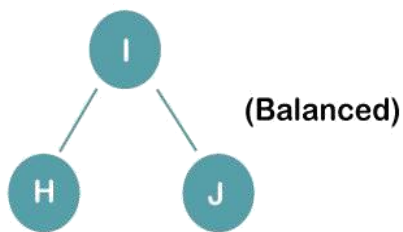


Q: Construct an AVL tree having the following elements H,I,J,B,A,E,C,F,D,G,K,L

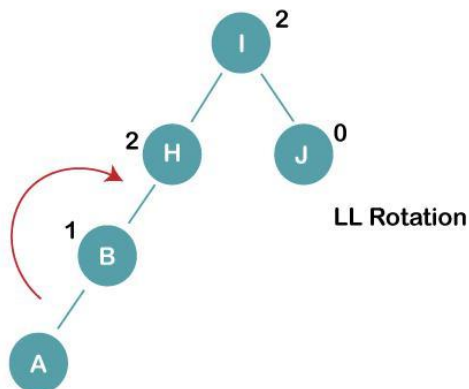
1. Insert H, I, J



On inserting the above elements, especially in the case of H, the BST becomes unbalanced as the Balance Factor of H is -2. Since the BST is right-skewed, we will perform RR Rotation on node H. The resultant balance tree is:

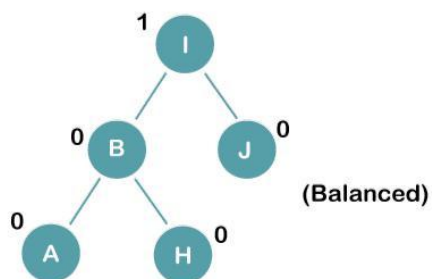


2. Insert B, A

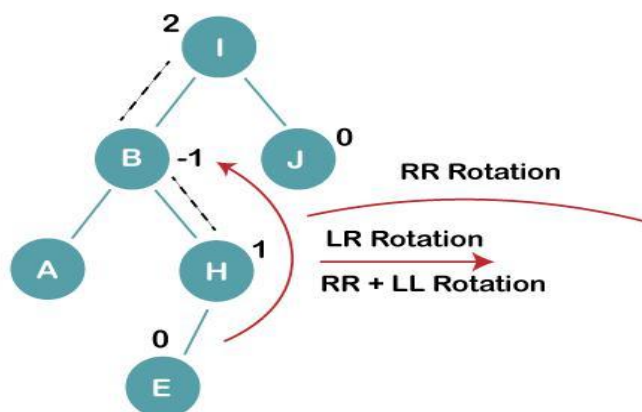


On inserting the above elements, especially in case of A, the BST becomes unbalanced as the Balance Factor of H and I is 2, we consider the first node from the last inserted node i.e. H. Since the BST from H is left-skewed, we will perform LL Rotation on node H.

The resultant balance tree is:



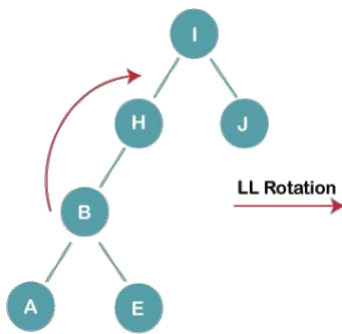
3. Insert E



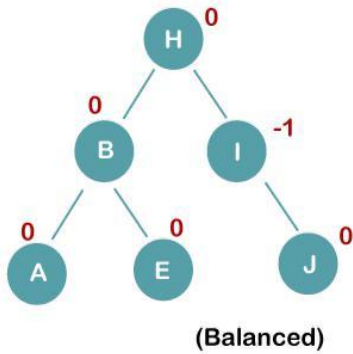
On inserting E, BST becomes unbalanced as the Balance Factor of I is 2, since if we travel from E to I we find that it is inserted in the left subtree of right subtree of I, we will perform LR Rotation on node I. LR = RR + LL rotation

3 a) We first perform RR rotation on node B

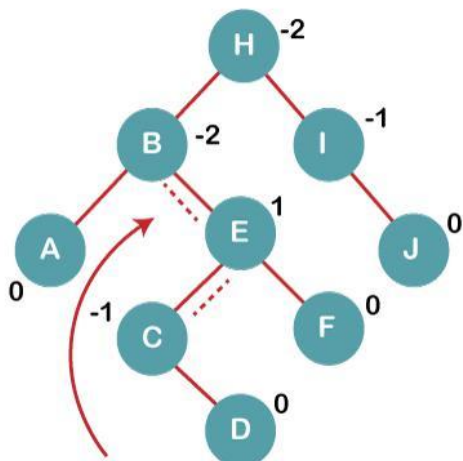
The resultant tree after RR rotation is:



3b) We first perform LL rotation on the node I
The resultant balanced tree after LL rotation is:

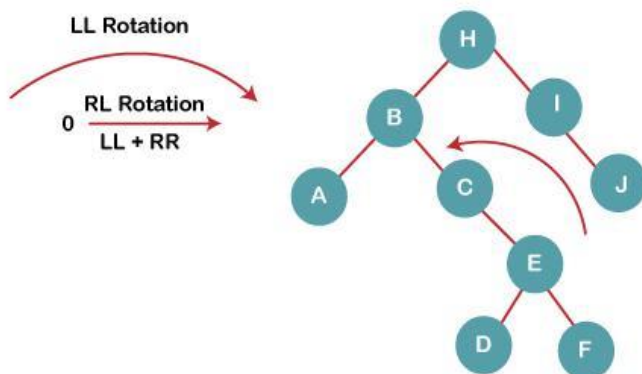


4. Insert C, F, D

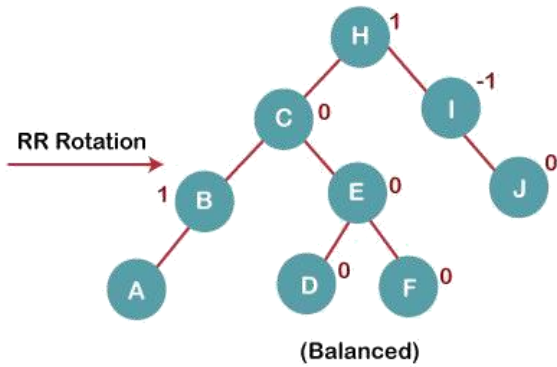


On inserting C, F, D, BST becomes unbalanced as the Balance Factor of B and H is -2, since if we travel from D to B we find that it is inserted in the right subtree of left subtree of B, we will perform RL Rotation on node I. RL = LL + RR rotation.

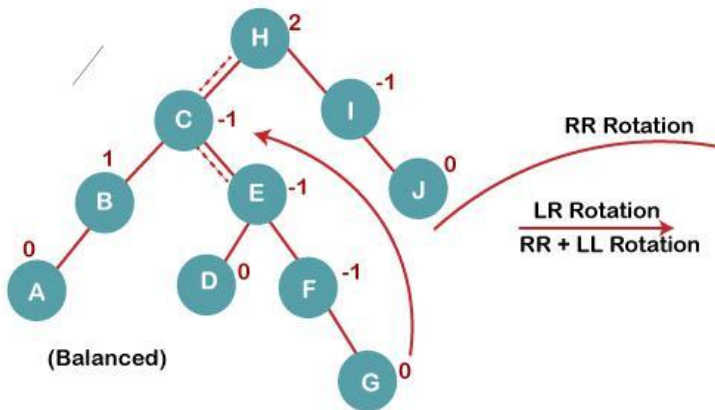
4a) We first perform LL rotation on node E
The resultant tree after LL rotation is:



4b) We then perform RR rotation on node B
The resultant balanced tree after RR rotation is:

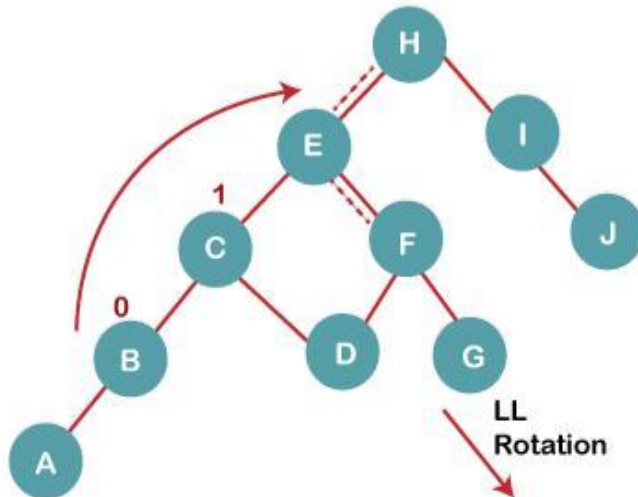


5. Insert G

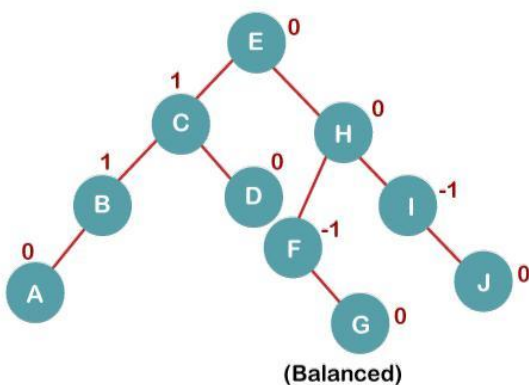


On inserting G, BST become unbalanced as the Balance Factor of H is 2, since if we travel from G to H, we find that it is inserted in the left subtree of right subtree of H, we will perform LR Rotation on node I. LR = RR + LL rotation.

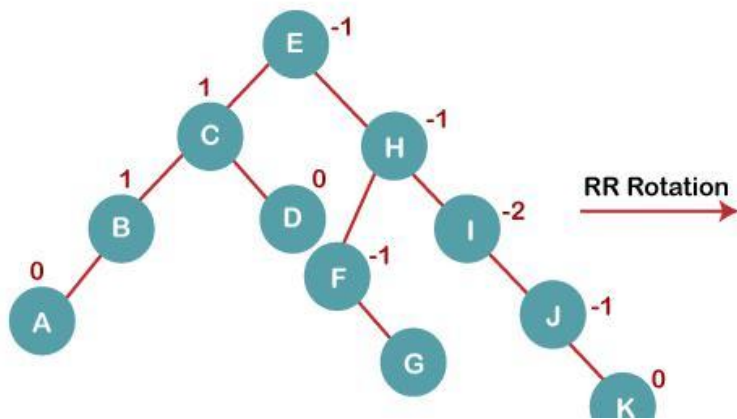
5 a) We first perform RR rotation on node C
The resultant tree after RR rotation is:



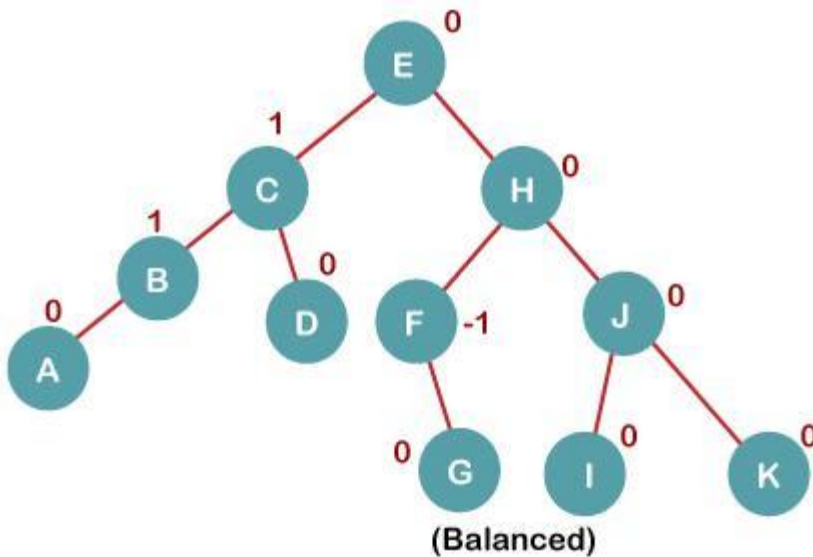
5 b) We then perform LL rotation on node H
The resultant balanced tree after LL rotation is:



6. Insert K

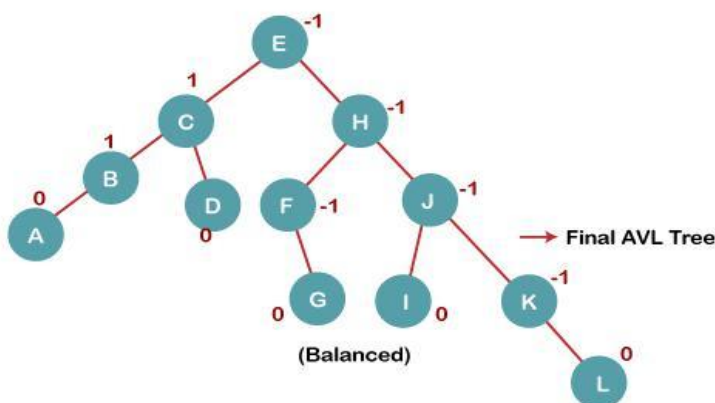


On inserting K, BST becomes unbalanced as the Balance Factor of I is -2. Since the BST is right-skewed from I to K, hence we will perform RR Rotation on the node I. The resultant balanced tree after RR rotation is:



7. Insert L

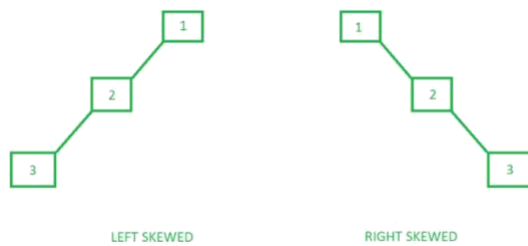
On inserting the L tree is still balanced as the Balance Factor of each node is now either, -1, 0, +1. Hence the tree is a Balanced AVL tree



Height balancing of tree is required : Height balancing of tree is required to implement an AVL tree. Each node must contain a balance factor, which indicates its states of balance relative to its sub-tree. .

Skewed Binary Tree:

If a tree which is dominated by left child node or right child node, is said to be a Skewed Binary Tree. In a skewed binary tree, all nodes except one have only one child node. The remaining node has no child.



In a left skewed tree, most of the nodes have the left child without corresponding right child.

In a right skewed tree, most of the nodes have the right child without corresponding left child.

B - Tree:

A B-tree is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic time.

B trees are extended binary search trees that are specialized in m-way searching, since the order of B trees is 'm'. Order of a tree is defined as the maximum number of children a node can accommodate. Therefore, the height of a b tree is relatively smaller than the height of AVL tree.

The main difference between a binary search tree and a B-tree is that a B-tree can have multiple children nodes for a parent node. However, a binary search tree can contain only two children nodes for a parent node.

Unlike a binary search tree, a B-tree can have multiple keys in a node. The number of keys and the number of children for any node depends on the order of the B-tree.

A B-tree of order m is a tree which satisfies the following properties :

- Every node has at most m children.
- Every non-leaf node (except root) has at least $m/2$ children.
- The root has at least two children if it is not a leaf node.
- A non-leaf node with k children contains $k - 1$ keys.
- All leaves appear in the same level.

Application of B-tree :

Here, are reasons of using B-Tree

- Reduces the number of reads made on the disk
- B Trees can be easily optimized to adjust its size (that is the number of child nodes) according to the disk size
- It is a specially designed technique for handling a bulky amount of data.
- It is a useful algorithm for databases and file systems.
- A good choice to opt when it comes to reading and writing large blocks of data

B-tree Operations:

Three primary operations can be performed on a B-tree: searching, insertion, and deletion.

1. Searching: The structure of the B-tree is similar to the binary search tree with some added properties. Hence, the searching operation in a B-tree works the same as a BST. Initially, we start searching from a root node.

We either go to the left or right subtree from the root node, depending on the key value of the node we want to search. On top of this, in a B-tree, we have several decisions as there can be more than two branches for a node.

The searching time complexity of a B-tree in the best, as well as the worst-case, is $O(\log n)$, where n denotes the total number of elements in a B-tree.

2. Insertion: Insertion is the operation in which we insert or add elements in a B-tree. During this process, we need to keep some points in mind. We can't insert any element at the root node of a B-tree. We have to start inserting elements from the leaf node.

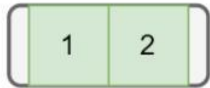
Additionally, while inserting a node in a B-tree, we must check the maximum number of keys at any node and add elements in ascending order.

Let's take the example of inserting some nodes in an empty B-tree. Here, we want to insert 10 nodes in a B-tree: (1,2,3,4,5,6,7,8,9,10), with the order of B-tree is 3.

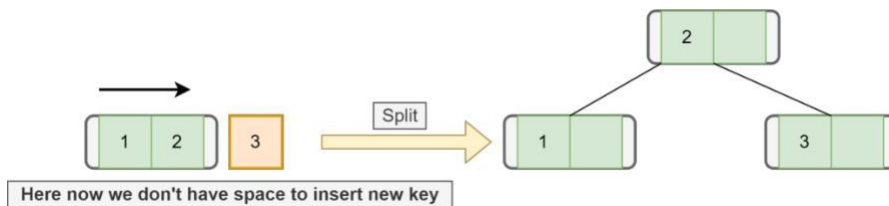
Hence, the maximum number of children a node can contain is 3. Additionally, the maximum and the minimum number of keys for a node, in this case, would be 2 and 1 (rounded off).

While inserting any element, we must remember that we can only insert values in ascending order. Moreover, we always insert elements at the leaf node so that the B-tree always grows in the upward direction.

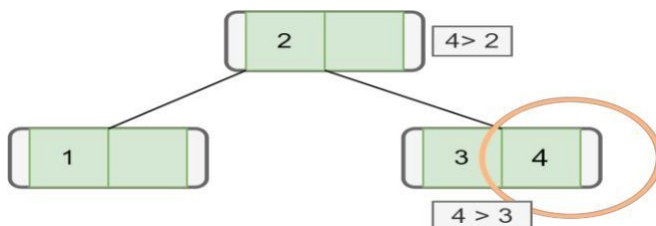
Let's start the insertion process with nodes 1 and 2 :



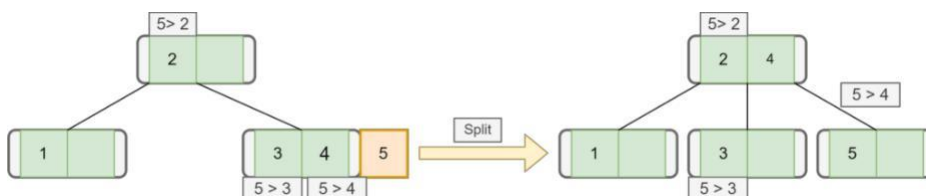
Next, **we want to inset node 3**. But there's no space for inserting new elements in that leaf node as the maximum key a node can contain is here. Therefore, we need to split the node. To split the node, we take the median key and move that key upward:



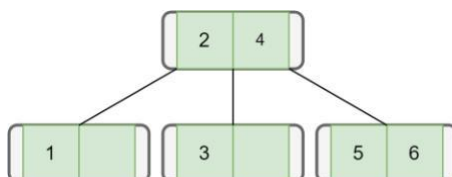
Now **we want to insert the next element, which is 4**. As 4 is greater than 2, we first go to the right subtree. In the right subtree, there's a node with 3. The key of the incoming node is greater than 3. Hence, we'll add this node to the right of 3:



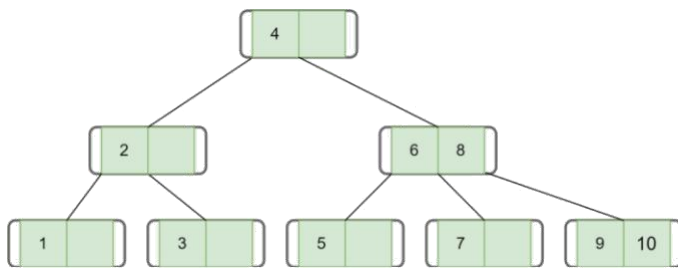
Let's insert the next node with key 5. We first check the root, and as 5 is greater than 2, we go to the right subtree. In the right subtree, we see 4 is greater than 3 and 4. Hence, we insert on the right side of 4. But again there's not enough space. Therefore, we split it and move the median of that is 4, to the upper node:



Similarly, **we insert our next node with key 6**:



Finally, let's add all the remaining nodes one by one following the properties of the B-tree:

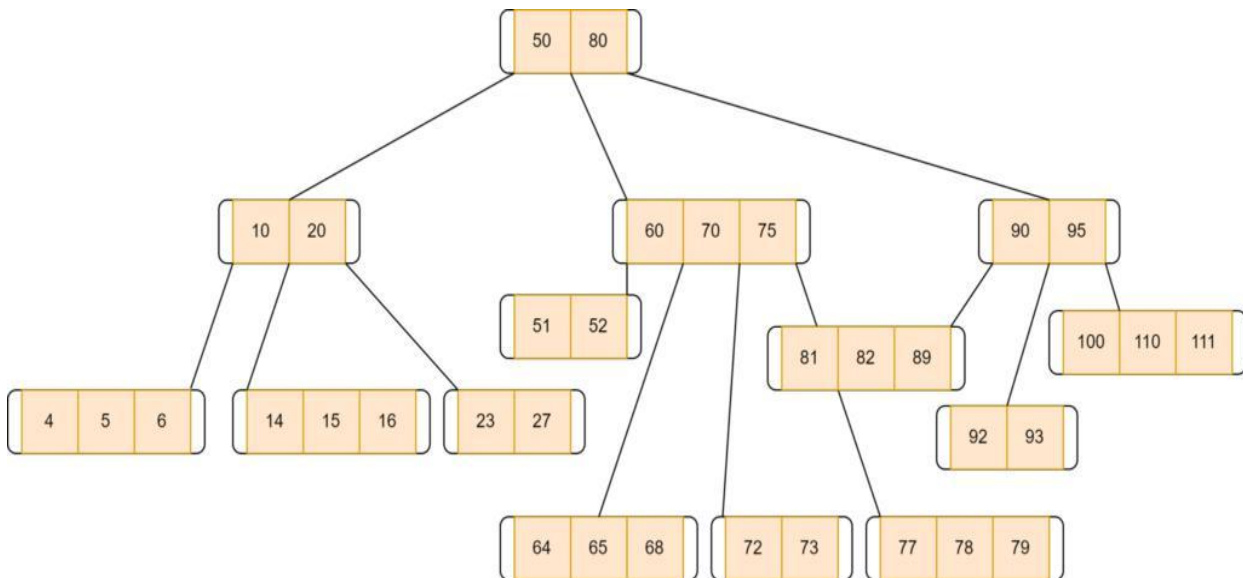


The time complexity of the insertion process of a B-tree is **$O(\log n)$** .

3. Deletion: Deletion is the process in which we remove keys from a B-tree. During this process, we need to maintain B-tree properties. After deleting any key, the tree must be arranged again to follow B-tree properties. Additionally, we need to search that key in the B-tree before deletion.

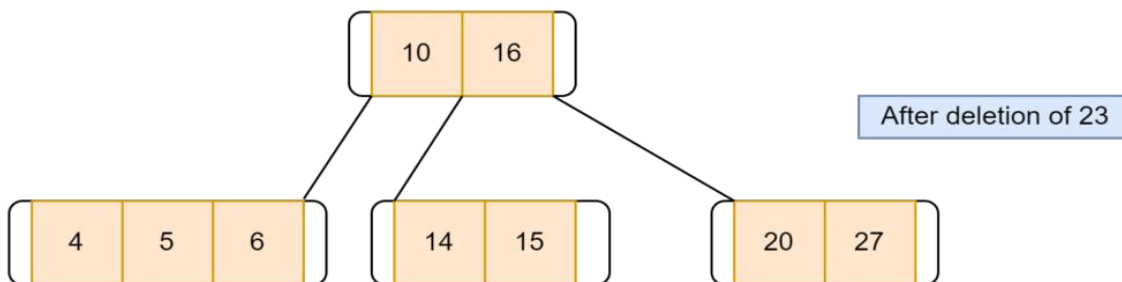
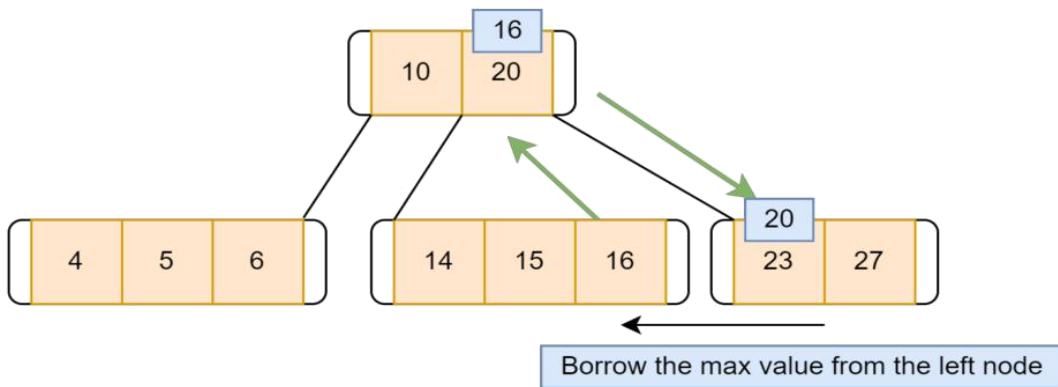
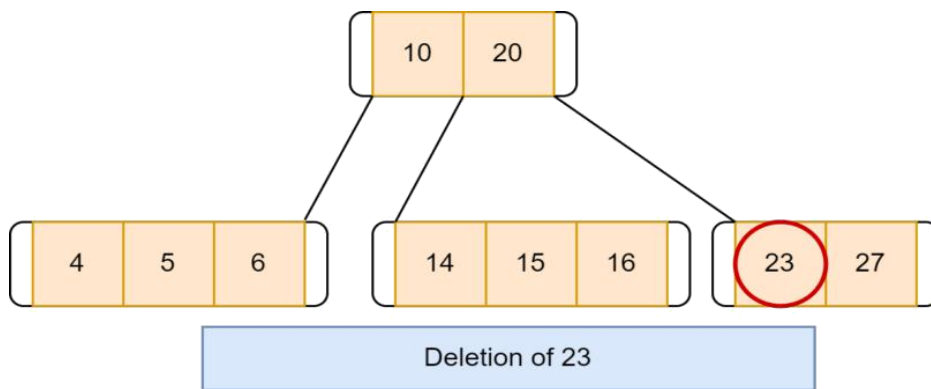
There can be two possible situations while deleting a key from a B-tree. Firstly, the key we want to delete is in a leaf node. Secondly, the key we want to delete is in the internal node.

Let's take an example of deletion from the B-tree: Here, we're taking a B-tree of order 5. The maximum number of children a node can contain is 5. Additionally, the maximum and the minimum number of keys for a node, in this case, would be 4 and 2:

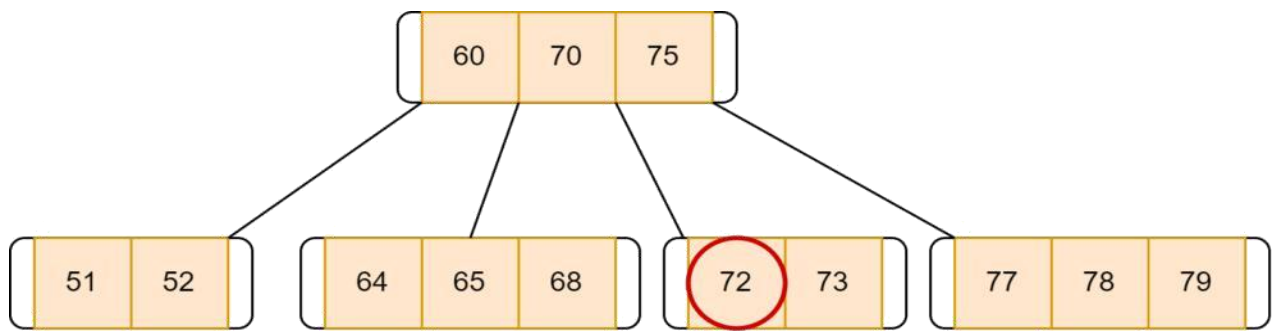


If the target key? leaf node contains only the minimum number of keys, we can't simply delete the key. Because if we do so, it'll violate the condition of the B-tree. In such a case, borrow the key from the immediate left node if that node has more than the minimum required keys.

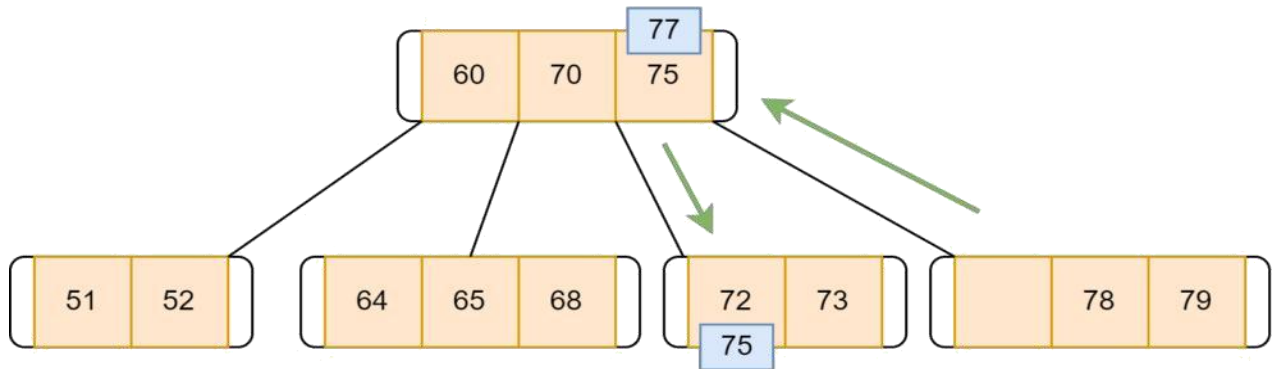
We'll transfer the maximum value of the left node to its parent node. After that, the key with greater value will be transferred to the borrower node. Moreover, we can remove the target key from the node. Let's say we want to remove a node with key 23:



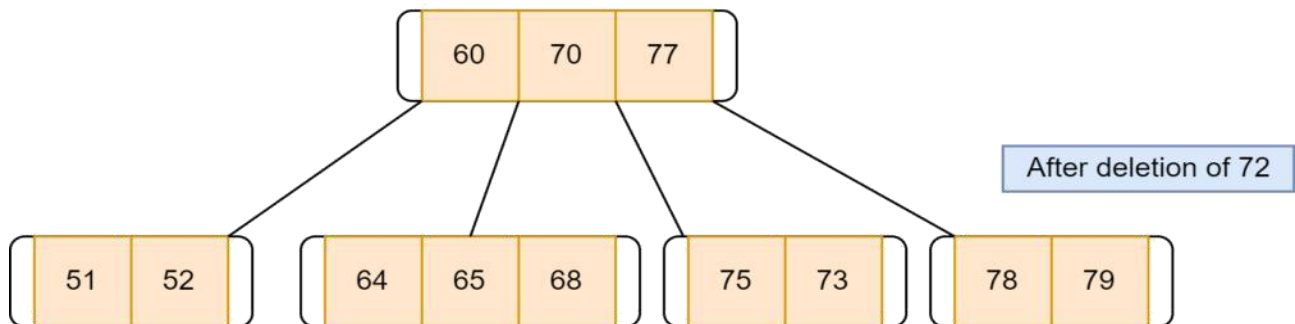
Another possible way is to borrow the key from the immediate right node if that node has more than the minimum required keys. We'll transfer the minimum value of the right node to its parent node. The key value, which is smaller, will be transferred to the borrower node. After that, we can remove the target key from the node. Let's say we want to remove a node with key 72:



Deletion of 72

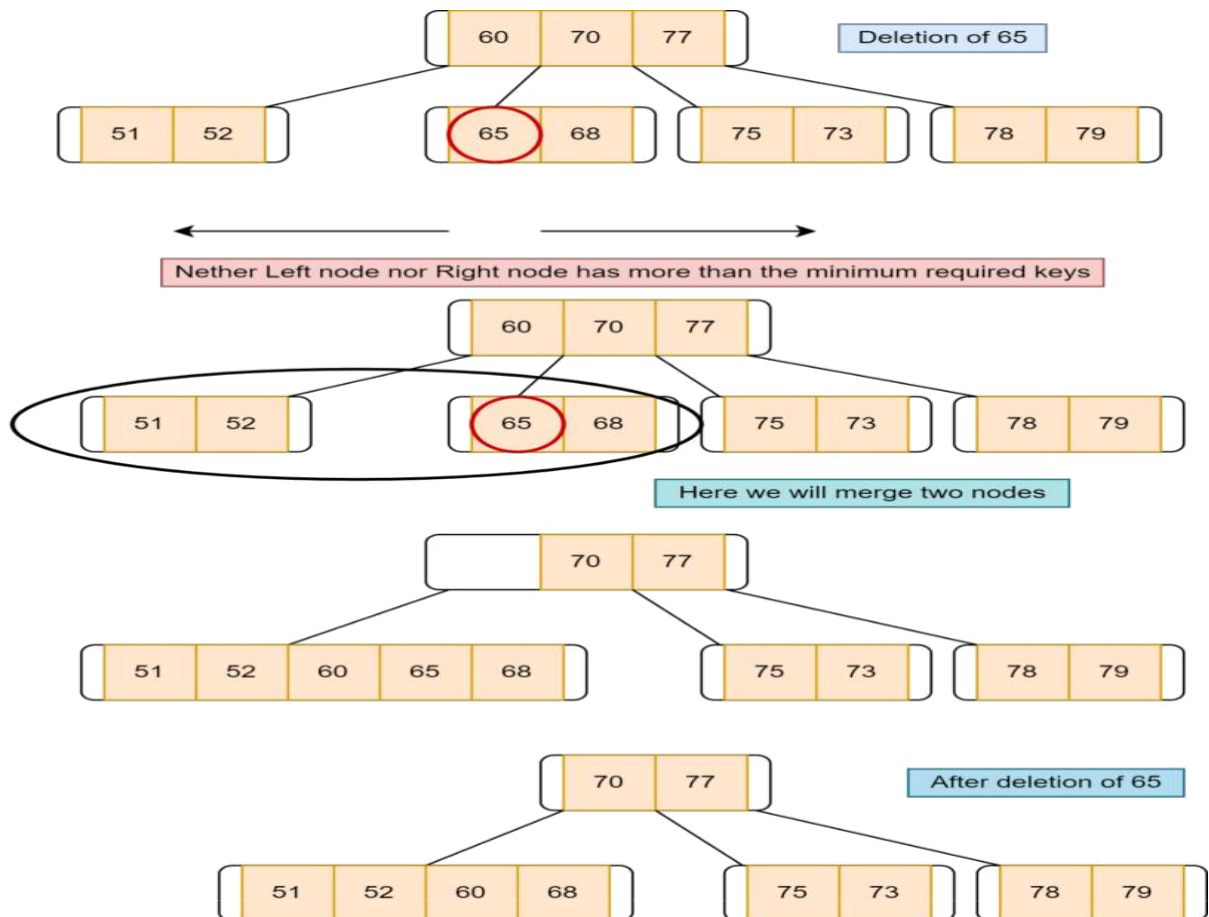


Borrow the minimum value from the right node

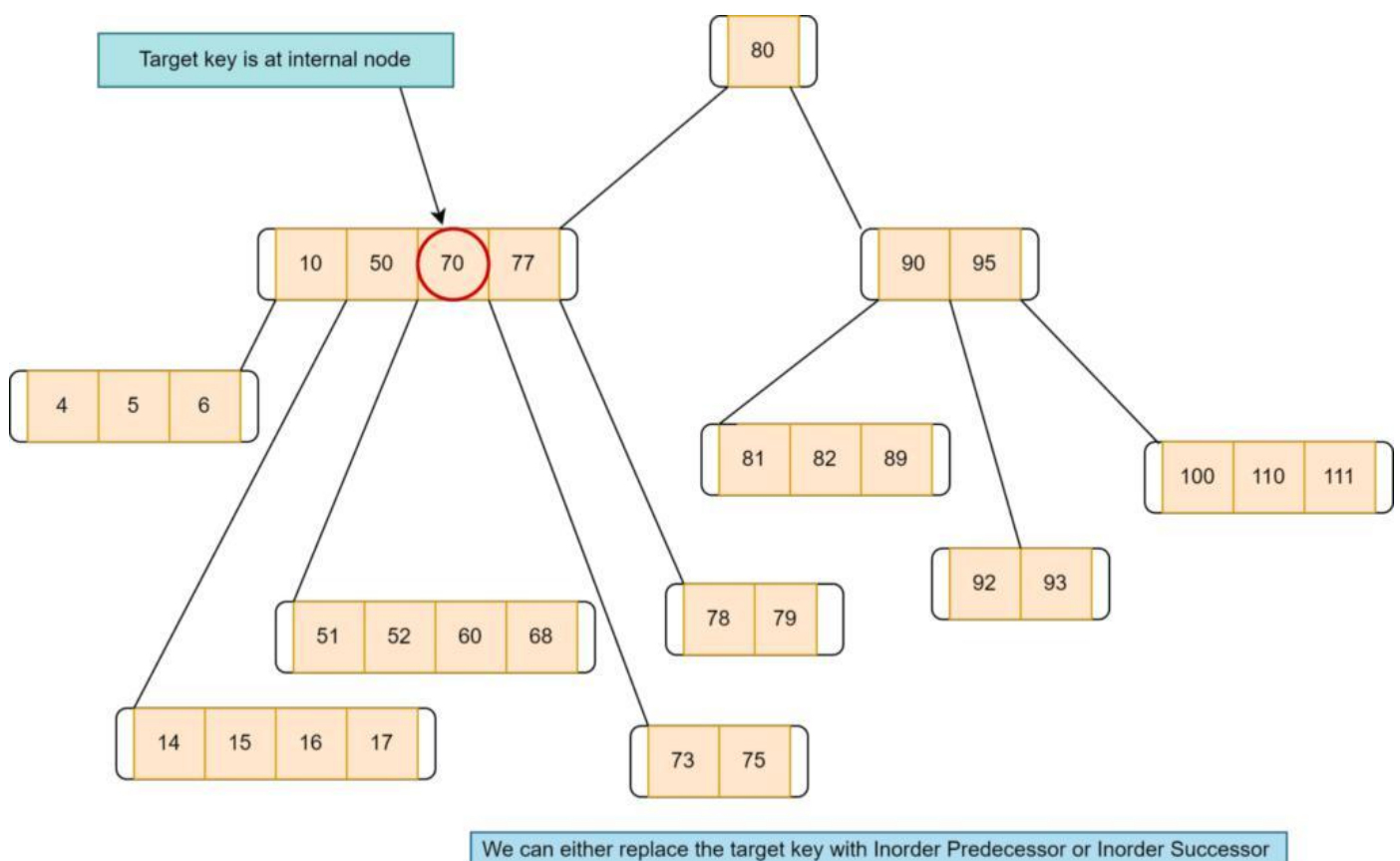


Now let's discuss the scenario when neither the left sibling nor the right sibling of the target key's node has more than the minimum required keys. In such a case, we need to merge two nodes. Out of two nodes, one node should contain our target node's key. While merging, we also need to consider the parent nodes as well.

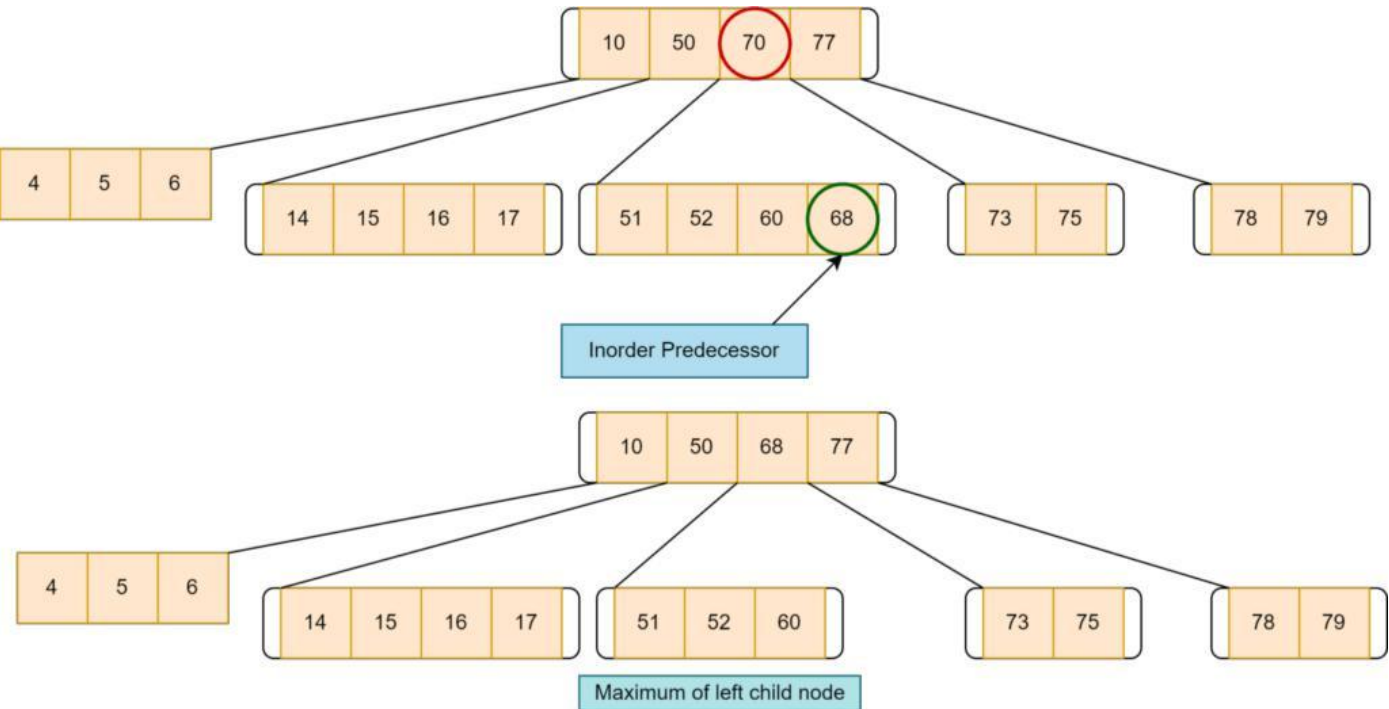
Suppose we want to delete a node with a key of 65. Its sibling nodes don't contain more than the minimum number of keys. Here the first step is merging the nodes. In this example, we have merged the left node of the target key's node. After merging the nodes, we delete our target node:



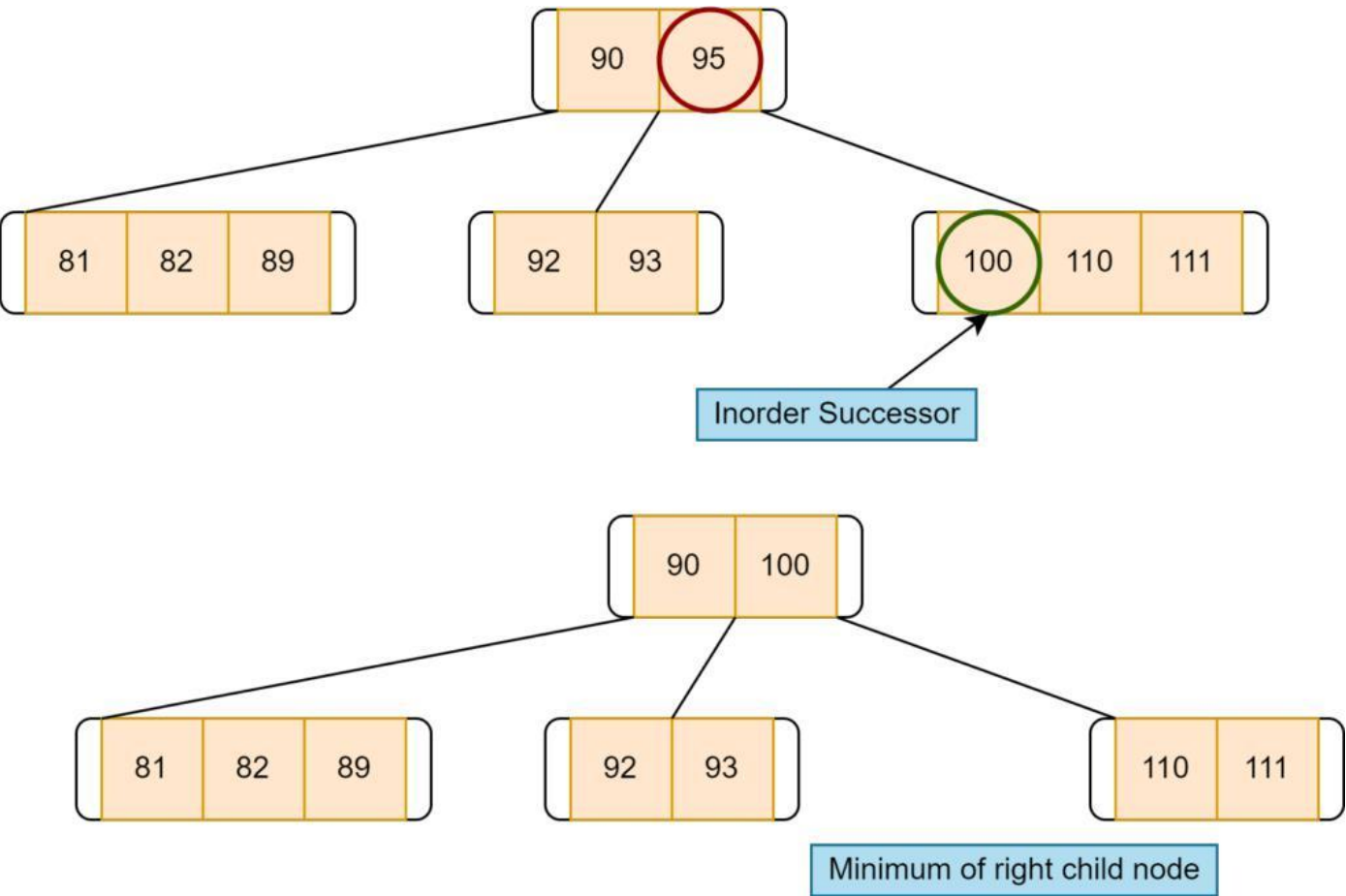
Now let's discuss the case when the target key is at an internal node:



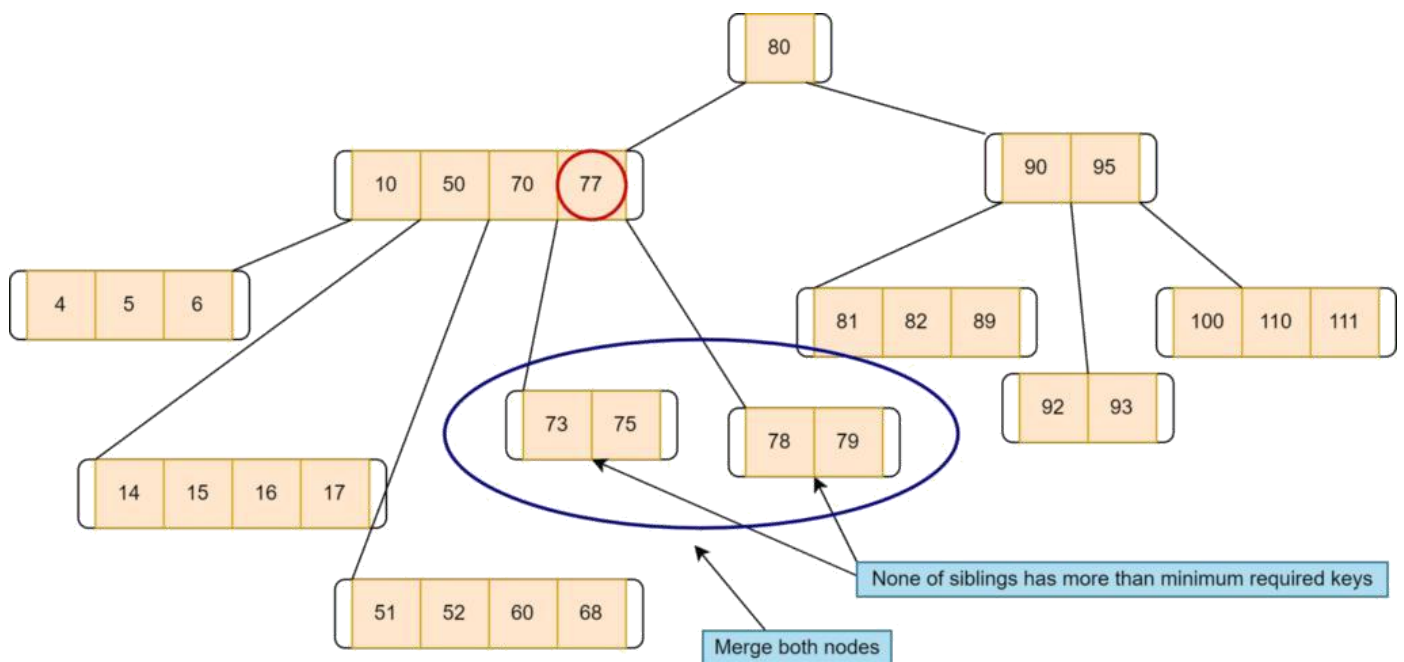
In such a case, **the first possible option is to replace the target key with its inorder predecessor**. Here we take the left node of the target key, select the highest value key, and replace that with the target key. Here we want to delete node 70:



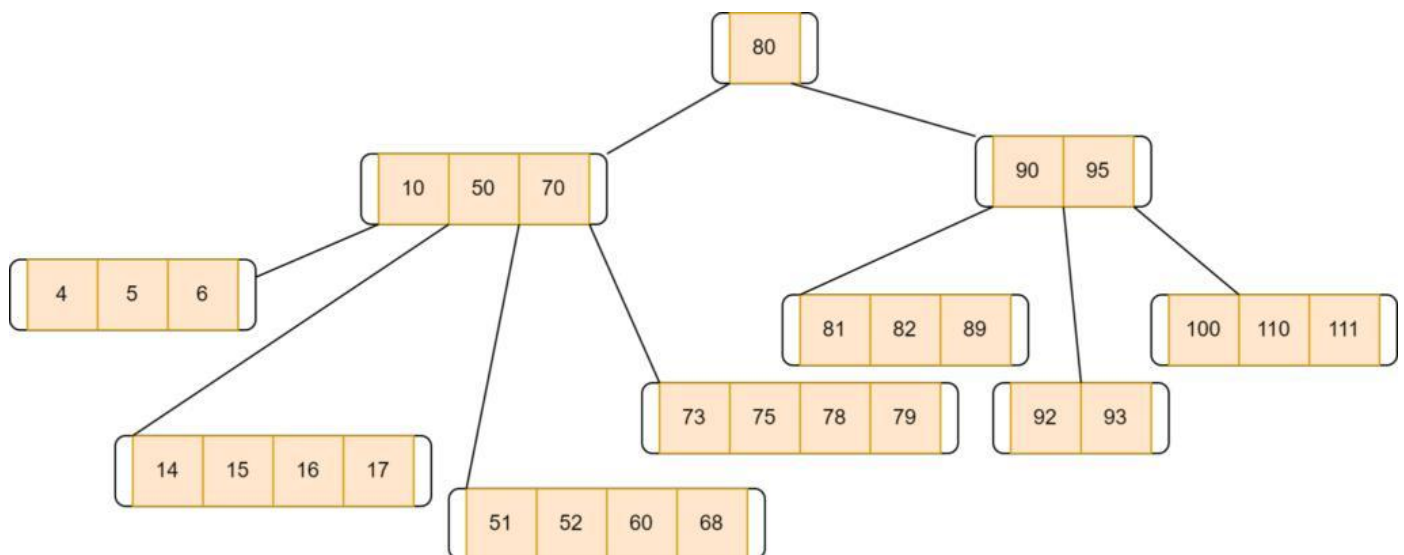
If the left doesn't have more than the minimum required keys, we replace the target key with its **inorder successor**. Here, we'll take the right node of the target key, select the lowest value key and replace that with the target key:



If the target key's inorder successor node and inorder predecessor node don't have more than the minimum number of the required key, we need to merge two adjacent nodes. **For example, deletion of 77:**



After deletion of 77:



The time complexity of the deletion process of a B-tree is $O(\log n)$.

Binary Heaps. Heap is a fundamental data structure in computer science that organizes data in a binary tree format with specific properties. It is characterized by its ability to efficiently retrieve the minimum or maximum element from a set of values. Two variations of heap, known as Max Heap and Min Heap, play a crucial role in various algorithms and applications.

Min Heap and Max Heap

Heap is a tree-based data structure that satisfies the property of complete binary tree where the key at the root must be minimum or maximum. The storage of heap is done on contiguous memory locations unlike general trees.

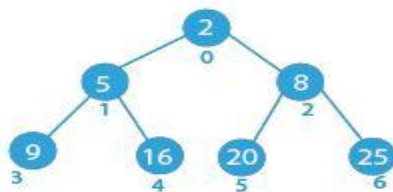
0	1	2	3	4	5	6
2	5	8	9	16	20	25
Left		$2i + 1$				
Right		$2i + 2$				
Parent		$\left\lfloor \frac{i-1}{2} \right\rfloor$				

While the storage is done contiguously, the parent and child nodes (both left and right) across the levels of tree can be accessed using the formula mentioned above.

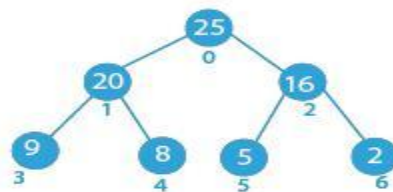
Min Heap :- The smallest element is located at the root of the tree in the min heap such that it is easier to pick the smallest element when heap pop is performed.

Max Heap :- The greatest element is located at the root of the tree in the max heap such that it is easier to pick the largest element when heap pop is performed.

The illustration stating the both is represented below:



Min Heap



Max Heap

Difference between Max Heap and Min Heap

Max Heap	Min Heap
The data at the root node should be greater than each of the child nodes.	The data at the root node should be smaller than each of the child nodes.
The element having the highest value has the highest priority assigned to it.	The element having the lowest value has the highest priority assigned to it.
The first value to be extracted is the maximum value.	The first value to be extracted is the minimum value.
It is used for the purpose of implementing Priority Queue.	It is used for the purpose of implementing Dijkstra Graph Algorithm and Minimum Spanning Trees.
It is used to sort the array in ascending order using Heap Sort.	It is used to sort the array in ascending order using Heap Sort.
Operations performed in Max Heap include Extract Maximum, Get Maximum and Insertion.	Operations performed in Min Heap include Extract Minimum, Get Minimum and Insertion.
The root of the tree must have the maximum value.	The root of the tree must have the minimum value.

Heap sort

1. Heap sort finds the largest element and puts it at the end of array, then the second largest item is found and this process is repeated for all other elements.
2. The general approach of heap sort is as follows :
 - a. From the given array, build the initial max heap.
 - b. Interchange the root (maximum) element with the last element.
 - c. Use repetitive downward operation from root node to rebuild the heap of size one less than the starting.
 - d. Repeat step (a) and (b) until there are no more elements.

Analysis of heap sort : Complexity of heap sort for all cases is $O(n \log^2 n)$.

HEAP-SORT(A) :

1. BUILD-MAX-HEAP (A)
2. for $i = \text{length}[A]$ down to 2
3. do exchange $A[1] \leftrightarrow A[i]$
4. $\text{heap-size}[A] = \text{heap-size}[A] - 1$
5. MAX-HEAPIFY (A, 1)

BUILD-MAX-HEAP(A)

1. $\text{heap-size}[A] = \text{length}[A]$
2. for $i = \text{length}[A]/2$ down to 1
3. do MAX-HEAPIFY(A, i)

MAX-HEAPIFY (A, i) :

1. $l = \text{left}[i]$
2. $r = \text{right}[i]$
3. if $l < \text{heap-size}[A]$ and $A[l] > A[i]$
4. then $\text{largest} = l$
5. else $\text{largest} = i$
6. if $r < \text{heap-size}[A]$ and $A[r] > A[\text{largest}]$
7. then $\text{largest} = r$
8. if $\text{largest} \neq i$
9. then exchange $A[i] \leftrightarrow A[\text{largest}]$
10. MAX-HEAPIFY [A, largest]

Program:

```
#include <stdio.h>
```

```
void heapsort (int a[], int length)
```

```
{
    buildmaxheap(a, length);
    int heapsize, i, temp;
    heapsize = length - 1;
    for( i=heapsize; i >= 0; i--)
    {
        temp = a[0];
        a[0] = a[heapsize];
        a[heapsize] = temp;
```

```

heapsize--;
maxheap(a, 0, heapsize);
}
for( i=0; i < length; i++)
{
printf("\t%d" ,a[i]);
}
}

```

```

void buildmaxheap (int a[], int length)

```

```

{
int i, heapsize;
heapsize = length - 1;

for( i=(length/2); i >= 0; i--)

{
maxheap(a, i, heapsize);
}
}

```

```

void maxheap(int a[], int i, int heapsize)

```

```

{
int l, r, largest, temp;
l = 2*i;
r = 2*i + 1;
if(l <= heapsize && a[l] > a[i])
{
largest = l;
}
else
{
largest = i;
}
if( r <= heapsize && a[r] > a[largest])
{
largest = r;
}
if(largest != i)
{
temp = a[i];
a[i] = a[largest];
a[largest] = temp;
maxheap(a, largest, heapsize);
}
}

```

```

int main()

```

```

{
int a[10], i, size;
printf("Enter size of list"); // less than 10, because max size of array is 10 +
scanf("%d",&size);
printf( "Enter elements");

```



```
for( i=0; i < size; i++)  
{  
scanf("%d",&a[i]);  
}  
heapsort(a, size);  
return 0;  
}
```

*****END OF UNIT 4*****